Department of Computer Science and Engineering

Course Code & Name	DSA01 – Object-Oriented Programming with C++
Assessment	CO1 – Assignment 1 (individual, 50 marks): Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System
Course Outcome Assessed	CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs (K5 – Evaluate).
Student Reg. No.	192473003
Student Name	Saran. S

DSA01 ASSIGNMENT 1 – CO1 REPORT

Scenario – Fare Logic of the Namma Metro Smart-Card System

This report covers the fare software of a three-zone city metro: a first version that was written in a hurry and went wrong in four ways, and a rebuilt version that is exact, tested and then changed to meet new rules (a fourth zone, a weekend discount and monthly passes). Every task follows the same chain: **understand** the problem, **choose** the C++ features, **build** the program, **test** it with real runs and **compare** it with an alternative. Every output, warning and timing quoted here comes from a program in Appendix A that I compiled and ran myself; nothing is copied from memory.

Given information:

- **Nature of assignment:** individual, 1 week, 50 marks; Tasks 1–11 carry 42 marks and the Day-4 design challenge (Task 12) carries 8. Every task is at K5 (Evaluate) level.
- **Experimental environment:** g++ 13.3.0 on Ubuntu 24.04, x86-64. All programs are built with `g++ -std=c++17 -Wall -Wextra`; only the timing program uses `-O2`.
- **Primary evidence:** program output, compiler messages (errors and warnings), exact paise totals, and timings in nanoseconds per operation (best of 5 runs, `std::chrono::steady_clock`).
- **Data:** the 3,000,000 journeys of Task 10 come from `std::mt19937` with seed 2026, so every run gives the same journeys. No real passenger data is used.
- **Reproducibility:** the source code in Appendix A regenerates every table; the repository layout and the README are in Appendix B.

(a) Understand the Assignment

The assignment gives one scenario and twelve tasks. The tasks are not twelve separate problems: they walk through one fare program, from deciding what the objects are (Task 1) to changing the design when the rules change (Task 12). The table shows what each task asks and what I produced for it.

Task	What it asks	C++ concept	Evidence produced
1 (4)	Objects, OOP principles, rules	Classes, encapsulation, polymorphism	UML diagram, rule table, procedural comparison
2 (3)	Constants and tokens	constexpr, enum class, tokens	Macro trap output, compiler errors, identical assembly
3 (4)	Data types and ranges	sizeof, climits, long long	Type table, double error growth, int overflow
4 (4)	Implicit conversions	Integer division, sign-compare, narrowing	Prediction, real output, warnings, brace-init errors
5 (4)	Expressions and precedence	Operators, ++, %, ?:	Prediction, real output, operator table
6 (4)	Control structures	switch, loops, break/continue	Pseudocode, completed code, 10 AM / 8 PM check
7 (4)	Procedural against OO	Class with private data	Two programs, compile error for private data
8 (4)	Safe input and output	cin.fail, iomanip, getline/stoi	Runs with bad input, formatted report
9 (4)	Implementation and tests	Table-driven tests	29 test rows, mutation test, hand total
10 (4)	Expectation against execution	Overflow, precision, warnings	Totals for 20,000 / 500,000 / 3,000,000 journeys
11 (4)	Final design	Design defence	Evidence table, config-file alternative
12 (8)	Re-evaluate for new rules	Class hierarchy, tables	Impact analysis, tests, regression, cost of virtual calls

Table 1: Scope of the twelve tasks and the evidence produced for each.

Method used for every task. The same five steps are repeated so that each decision can be traced back to evidence:

- **Understand** the requirement and write down every assumption, because the scenario states fares but not concessions or weekends.
- **Choose** the C++ feature that fits and name the alternatives I rejected.
- **Build** a small program for each claim (the programs are listed in Appendix A).
- **Test** it: compile with `-Wall -Wextra`, run it, and keep the real output. For the two Predict-the-Output programs the prediction was written first.
- **Compare** with the alternative concept the task names, using Big-O and my own measurements.

Assumptions used throughout. The scenario leaves these open, so I fixed them once and kept each in one constant:

No.	Assumption	Reason
A1	Peak time is 08:00 up to 09:59 and 17:00 up to 19:59; so 10:00 and 20:00 are off-peak.	Both ends of a window are treated the same way (start included, end excluded). This removes the 10 AM / 8 PM inconsistency.
A2	Students get 25% off the base fare and senior citizens 50%.	The scenario says “a concession” but gives no figure. The two figures are one constant each.
A3	Order of rules: concession first, then the 20% peak surcharge. Percentages are taken in whole paise and rounded half-up.	Rounding at each step keeps every fare a whole number of paise.
A4	The fare depends only on the farthest zone reached.	Stated in the scenario.
A5	A journey is allowed only if balance − fare ≥ Rs 50 (5000 paise).	Stated in the scenario.

No.	Assumption	Reason
A6	A recharge must be a positive amount.	A negative or zero recharge is one way a balance could be corrupted.
A7	(Task 12) Weekend means Saturday and Sunday. The 10% discount replaces the peak surcharge and is taken after the concession.	“Does not combine with the peak surcharge” is read as: on a weekend there is no surcharge at all.
A8	(Task 12) A pass covers every journey whose farthest zone is one of its two zones, within one calendar month. A covered journey costs 0 and the Rs 50 test is not applied.	Needed to decide what “unlimited journeys in two zones” means at the gate.
A9	The SIMATS pseudocode standard was not included with the assignment, so my pseudocode uses a plain form: named algorithm, `<-` for assignment, indentation, `//` comments.	Re-check against the standard text before submitting.

Table 2: Assumptions A1–A9 (each one lives in a single constant or function).

Task 1 – Understand the Problem and Identify the Objects (4 Marks)

► Step 1: What was wrong with the first version

Before designing anything I rebuilt the hurried first version (`legacy_v0.cpp`) so that its defects could be seen and not just described. It stores rupees in a `double`, writes the fares inside three different functions and checks the balance with `balance > 0`. Its real output:

```
Inner Rs 10, Middle Rs 20, Outer Rs 30
revenue(2,1,1) = 70
10:00 peak? 1   20:00 peak? 0
single fares off by a paisa: 0 of 144
balance after an Outer peak journey from Rs 20 = -16
```

Reported symptom	What I found in the code	Evidence
10 AM peak, 8 PM not	The test is `hour >= 8 && hour <= 10` for the morning and `hour < 20` for the evening, so the two windows are not written the same way.	`10:00 peak? 1 20:00 peak? 0`
Negative balance	The gate only asks `balance > 0` before subtracting the fare, so a card with Rs 20 pays a Rs 36 fare.	Balance after the journey: −16
Fares typed in many places	`fareRs()`, `showTariff()` and `revenue()` each contain their own copy of Rs 10, 20, 30 (I counted 3 places).	A fare revision needs three edits and one can be forgotten
Fare off by a paisa	I could **not** reproduce this on a single fare: 0 of 144 zone/hour/concession combinations were wrong. With these rates every fare is a multiple of 50 paise, which `double` can hold exactly.	The error shows up in running totals instead (Task 3, Task 10)

Table 3: The four reported symptoms and what the reconstructed code shows.

The fourth row is a result I did not expect. The scenario says fares were off by a paisa, but with my assumed rates the single-fare arithmetic in `double` is exact. I kept the rule “no `double` for money” anyway, because the danger is latent: the moment a fare with a decimal part such as Rs 10.80 appears, `double` drifts (Figure 3). The three symptoms I could reproduce share one cause: nothing owns the money and the rules. The design therefore gives every value one owner.

► Step 2: The objects, their data and behaviour

Object	Data	Behaviour	Why it is an object
SmartCard	`balance_` (long long, paise, private)	`balance()`, `canTravel(f)`, `deduct(f)`, `recharge(a)`	It owns the money. If only the card can change the balance, the Rs 50 rule needs to be written in one place.
Passenger	`name_`, `category_`, `card_`	`name()`, `category()`, `card()`	The concession depends on who travels, not on the journey. A passenger has exactly one card.
Journey	zone, hour, minute	(plain data)	One journey is one unit of work at the gate; it carries the time that decides the surcharge.
Zone, Category	Inner/Middle/Outer; Regular/Student/Senior	(`enum class` values)	A value from a fixed set. A type of its own stops a zone being mixed up with an ordinary int.
FareTable	`BASE_PAISE[]`, percentages, peak limits	`isPeak()`, `concessionPercent()`, `fare()`, `pct()`	Every fare constant and the fare formula live here, so a revision is made in one place.

Table 4: Objects of the metro fare system, with the reason each one exists.

I did not make a `Gate` class. At this size the gate is only a loop that reads a record, asks `FareTable` for the fare and asks the card to deduct it (Figure 2). A class with no data of its own would only add a name. If gates later need their own state (a gate id, a day's count) it becomes an object then.

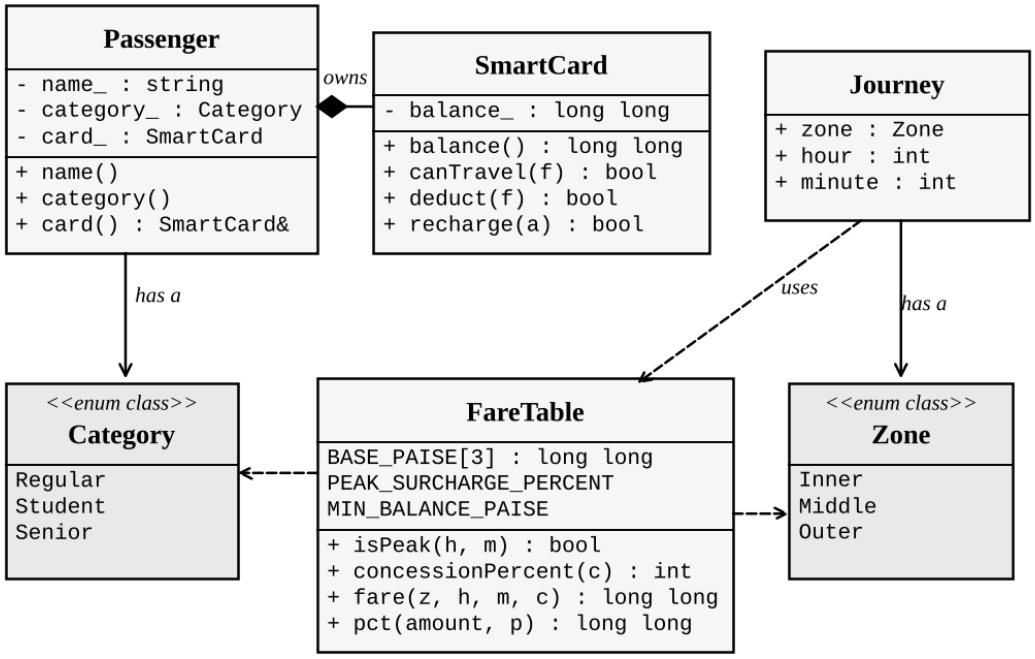


Figure 1: UML class diagram of the original design (solid diamond = owns, solid arrow = has a, dashed arrow = uses).

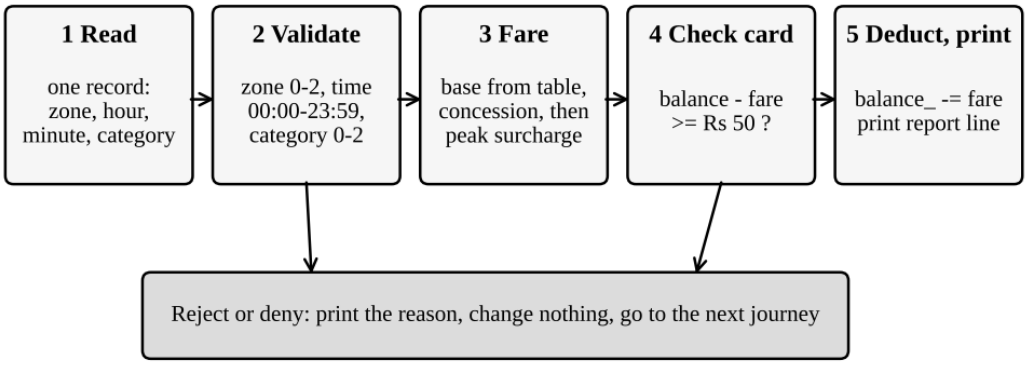


Figure 2: What the gate does for one journey; every failing step stops the pipeline without changing the card.

► Step 3: Procedural style and object-oriented style

In a procedural program the balance is a global variable and `fareOf()`, `canTravel()` and `deduct()` are free functions. The Rs 50 rule works only if every caller remembers to call `canTravel()` before `deduct()`. In the object-oriented program the data and the functions that may change it are one unit: `deduct()` makes the check itself, so the rule moves from the caller's memory into the card's code. Task 7 runs both and shows the difference.

► Step 4: The four OOP principles in this system

Principle	Where it appears here	What it buys
Encapsulation	`SmartCard::balance_` is private; the Rs 50 test sits in `canTravel()`	Code outside the card cannot create a negative balance (the compiler refuses, Task 7).
Abstraction	The gate calls `FareTable::fare(...)` and `card.deduct(f)` and knows nothing about percentages, rounding or zone tables	The gate loop does not change when the fare formula changes.
Inheritance	I first thought of `StudentPassenger` and `SeniorPassenger` subclasses and rejected it: only one number differs, so an `enum class Category` is simpler. Inheritance is used in Task 12 where it fits: `ConcessionRule`, `WeekendRule` and `PeakRule` derive from `Adjustment`.	Shared interface for pricing rules without copying code.
Polymorphism	`Adjustment::apply()` is virtual; `FareEngine` calls it without knowing which rule it is	A new rule is a new class, not an edit of the gate loop. Its cost is measured in Task 12.

Table 5: OOP principles and where each one appears.

► Step 5: Rules every correct journey must satisfy

No.	Rule	Where it is enforced	Checked by
R1	After the fare is deducted the balance is at least Rs 50, so it is never negative	`SmartCard::canTravel()` / `deduct()`	Task 9 rows 18–20
R2	Base fare is Rs 10, 20 or 30 by the farthest zone	`BASE_PAISE[]`	rows 1–3
R3	Peak is 08:00–09:59 and 17:00–19:59 only; the surcharge is exactly 20% of the concession-adjusted fare	`FareTable::isPeak()`, `fare()`	rows 4–13
R4	Students pay 75% and seniors 50% of the base fare (A2)	`concessionPercent()`	rows 14–17
R5	All money is whole paise in a `long long`; no floating point	types of all money variables	Task 3, Task 10
R6	A recharge must be positive; an unknown zone or a non-numeric hour is rejected, not guessed	`recharge()`, `toZone()`, `parseInt()`	rows 21–28
R7	Each fare constant is written once	`fare_core.h`	grep count (below)

Table 6: Rules every correct journey must satisfy, each with the place that enforces it.

Alternative Concept and Complexity Analysis

If the system were written in a purely procedural style (global variables and free functions) instead of `SmartCard`, `Journey` and `FareTable`, the solution would be:

```
ALGORITHM ProcessJourney_Procedural
  INPUT  : zone, hour, minute, category        // globals: balance, BASE[], MIN_BAL
  OUTPUT : gate decision, updated balance
  f <- fareOf(zone, hour, minute, category)    // free function reads BASE[]
  IF balance - f >= MIN_BAL THEN               // the CALLER must remember this test
      balance <- balance - f
      RETURN OPEN
  ELSE
      RETURN DENY
```

Question	Procedural	Object-oriented
Time per journey (measured, -O2)	1.72 ns	2.41 ns
Memory per card	8 bytes (`long long`)	8 bytes (`sizeof(SmartCard)` = 8)
Big-O	O(1) time, O(1) space per card	O(1) time, O(1) space per card
Places to edit when the minimum balance changes	one constant, but every function that touches `balance` must be re-read	one constant inside `SmartCard`; only its four member functions can use it
Places to edit when a fare changes	1 in `fare_core.h` (legacy version: 3)	1 in `fare_core.h`

Table 7: Procedural and object-oriented designs compared.

Both versions do the same fixed number of steps per journey, so time and memory are O(1) in both. The measured gap is below 1 ns per journey; I did not investigate its cause (the two fare functions are written slightly differently and are inlined differently), so I do not read it as a cost of OOP. The real difference is the number of places that can break a rule. A grep for the line `1000, 2000, 3000` over the final programs that share the header returns **one** line (in `fare_core.h`).

Result of Task 1: six objects with a reason each, seven rules each tied to code and a test, and a measured reason to prefer the class design: same speed, fewer places to go wrong.

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

► Step 1: The five kinds of tokens

I split two lines of `fare_core.h` into tokens: `constexpr long long MIN_BALANCE_PAISE = 5000;` and `if (isPeak(hour, minute)) f += pct(f, PEAK_SURCHARGE_PERCENT);`

Token kind	Examples from my program	Remark
Keywords	`constexpr`, `long`, `int`, `bool`, `enum`, `class`, `switch`, `case`, `if`, `return`, `static`	Reserved by C++; cannot be used as names
Identifiers	`Zone`, `BASE_PAISE`, `fare`, `canTravel`, `balance_`, `MIN_BALANCE_PAISE`	Names I chose
Constants (literals)	`5000`, `1000`, `20`, `100`, `50`, `true`	Integer and boolean literals
Operators	`=`, `+=`, `-=`, `*`, `/`, `%`, `>=`, `&&`, `||`, `::`	Specify computations
Punctuators	`{ } ( ) [ ] ; , :`	Group and separate; carry no value

Table 8: Token kinds with examples from the fare program.

► Step 2: #define, const and constexpr

Feature	#define	const	constexpr
Handled by	Preprocessor (text replacement)	Compiler	Compiler
Has a type	No	Yes	Yes
Obeys scope	No – applies from the #define onward, everywhere	Yes	Yes
Known at compile time	Yes (it is only text)	Only if the initialiser is constant	Always – the compiler checks
Usable as array size / case label	Yes	Sometimes	Yes, guaranteed
Seen in a debugger	No	Yes	Yes

Table 9: Three ways to name a constant.

I chose `constexpr` for the fare table because it is typed, scoped and checked at compile time. The macro form has two real traps, and I reproduced both. First, a macro is text: `#define FARE_INNER_MACRO 500 + 500` makes `FARE_INNER_MACRO * 2` expand to `500 + 500 * 2`.

```
macro      FARE_INNER_MACRO * 2 = 1500  (meant 2000)
constexpr  FARE_INNER_C * 2     = 2000
plain enum: OOuter + 5 = 7, OInner == 0 is 1
enum class: Zone::Outer needs a cast to print: 2
sizeof(OldZone) = 4, sizeof(Zone) = 1, sizeof(FARE_INNER_C) = 4
```

Second, a macro has no scope. `#define PEAK 20` replaces the word `PEAK` everywhere after it, even a member name in an unrelated struct, and the compiler error points at a number and not at the real mistake:

```
t2_err_macro.cpp:1:14: error: expected unqualified-id before numeric constant
t2_err_macro.cpp:1:14: error: expected unqualified-id before numeric constant
t2_err_macro.cpp:3:34: error: expected ‘;’ before numeric constant
```

► Step 3: enum and enum class for zones

A plain `enum` converts to `int` silently, so `OOuter + 5` compiles and gives 7 (see the output above). An `enum class` keeps its names inside their own scope (`Zone::Outer`) and refuses to turn into an int by itself:

```
t2_err_enum.cpp:2:15: error: cannot convert ‘Zone’ to ‘int’ in initialization
```

`enum class Zone : unsigned char` also takes 1 byte against 4 for the plain `enum` (`sizeof` output above), which is why `Journey` and `Category` stay small.

► Step 4: Naming rules and conventions

- **Rules:** letters, digits and underscore only; cannot start with a digit; case-sensitive; cannot be a keyword; names beginning with an underscore followed by a capital letter are reserved by the compiler.
- **Conventions I used:** constants in `UPPER_SNAKE_CASE` with the unit in the name (`MIN_BALANCE_PAISE`, so nobody has to guess rupees or paise); types in `PascalCase` (`SmartCard`); functions and variables in `camelCase` (`canTravel`); private data members end with an underscore (`balance_`).

► Step 5: The single set of constants

```
constexpr int ZONE_COUNT = 3;
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_PERCENT = 25;                                // assumption A2
constexpr int SENIOR_PERCENT = 50;                                 // assumption A2
constexpr long long MIN_BALANCE_PAISE = 5000;                      // Rs 50 must remain
constexpr int PEAK_AM_START = 8 * 60;                              // 08:00 included
constexpr int PEAK_AM_END = 10 * 60;                               // 10:00 excluded
constexpr int PEAK_PM_START = 17 * 60;                             // 17:00 included
constexpr int PEAK_PM_END = 20 * 60;                               // 20:00 excluded
static_assert(sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]) == ZONE_COUNT, "one fare per zone");
```

To revise a fare, the Rs 20 Middle fare for example, I edit one number in this block and rebuild. No other file contains a fare (Task 1 grep: 1 line).

Alternative Concept and Complexity Analysis

If the fares were `#define` macros and the zones plain integers, what changes is safety, not speed:

Point	`#define` + **plain integers**	`constexpr` + `enum class`
Type checking	None: `INNER + 5` is accepted	Typed; `Zone::Outer + 5` is an error
Scope	Global from the `#define` line (reproduced above)	Normal C++ scope
Brackets needed	Yes, or the `500 + 500` trap appears	No
Memory	No storage	Usually none; only a table used at run time (`BASE_PAISE`, 3 × 8 = 24 bytes, O(z) for z zones)
Running time	O(1), resolved at compile time	O(1), resolved at compile time
Generated code (`g++ -O2 -S`)	Assembly of the macro version and the constexpr version of the same function is **identical** (diff is empty)	same as left

Table 10: Macro-based constants against constexpr and enum class.

Result of Task 2: one block of `constexpr` constants and two `enum class` types. The two designs compile to the same machine code here, so the choice is about stopping mistakes at compile time.

Task 3 – Choose Data Types and Check Type Compatibility (4 Marks)

► Step 1: The type chosen for every value

`t3_types.cpp` prints `sizeof` and the limits from `<limits>` on my machine (g++ 13.3.0, 64-bit Linux):

```
type           B  min                   max                   used for
bool           1  0                     1                     peak flag
char           1  -128                  127                   category letter
unsigned char  1  0                     255                   Zone, Category (enum base)
int            4  -2147483648           2147483647            hour, minute, zone number
unsigned int   4  0                     4294967295            (not used for money)
long long      8  -9223372036854775808  9223372036854775807   fare, balance, totals (paise)
float          4  -3.403e+38            3.403e+38             (only in the experiments)
double         8  -1.798e+308           1.798e+308            (only in the experiments)
```

Value	**Type**	**Size**	**Why**
Fare, balance, recharge, daily total	`long long` (paise)	8 bytes	Exact whole numbers; range far above any network total (see Step 4)
Hour, minute, zone number	`int`	4 bytes	Small whole numbers; `int` is the natural arithmetic type
Peak flag, “allowed” result	`bool`	1 byte	Two outcomes
Zone, Category	`enum class … : unsigned char`	1 byte	Fixed small set of values
Journey count	`long long`	8 bytes	3,000,000 is inside `int`, but the totals that grow with it are not; one type avoids mixed arithmetic
Rupee value for display only	`double`, after dividing paise by 100.0	8 bytes	Never stored or added; shown with `fixed` and `setprecision(2)`
A computed type	`auto f = 7 / 2.0`	8 bytes	`auto` picked `double` from the expression; I use it only where the type is obvious

Table 11: Data-type decisions (sizes measured with sizeof).

► Step 2: Why money is stored as integer paise

A `double` stores numbers in binary, and decimals such as 0.1 or 10.8 have no exact binary form. The program printed:

```
0.1 + 0.2        = 0.30000000000000004
10.8 is stored as 10.800000000000001
ten times 0.10   = 0.99999999999999989  equals 1.00? false
3 * 0.1 == 0.3?  false
```

So `3 * 0.1 == 0.3` is false and ten additions of Rs 0.10 do not make Rs 1.00. An integer such as 1080 paise is always exact. The next table adds the same fare n times in `double` and compares the total with the exact integer-paise total:

n fares	Exact total, Rs 10.80 each (Rs)	Error, Rs 10.80 each (Rs)	Error, Rs 0.10 each (Rs)
10	108.00	-1.4211e-14	-1.1102e-16
1,000	10,800.00	-7.8217e-11	-1.4069e-12
100,000	1,080,000.00	1.9970e-06	1.8848e-08
1,000,000	10,800,000.00	6.4231e-05	1.3329e-06
3,000,000	32,400,000.00	1.5543e-03	1.9568e-06
10,000,000	108,000,000.00	-1.8909e-02	-1.6102e-04

Table 12: Absolute error of a double running total against exact integer paise (measured, t3_types.cpp).

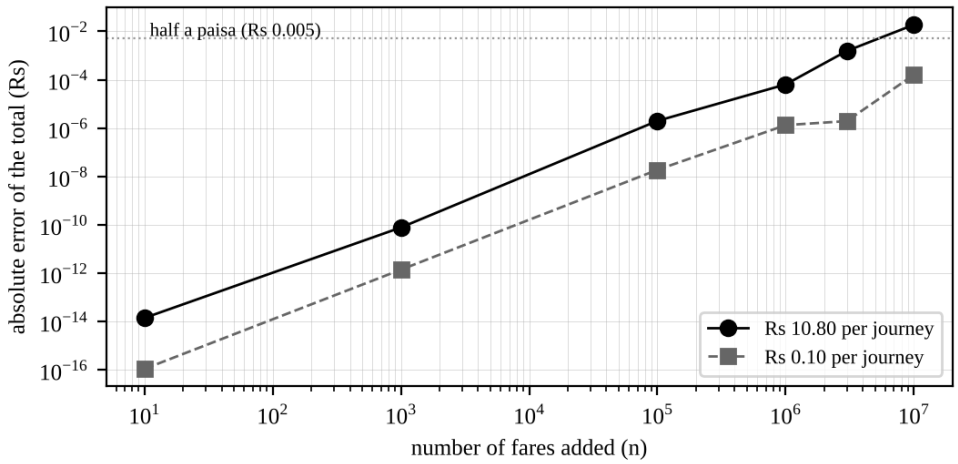


Figure 3: The error of a double total grows roughly with n; the integer-paise error is always 0.

The error reaches about Rs 0.019, nearly two paise, at ten million additions of Rs 10.80. That is the kind of difference that makes a day's collection fail to match the bank. The resolution is to keep every amount as `long long` paise and divide by 100 only when printing.

► Step 3: Type compatibility inside one expression

When types meet in one expression the compiler first promotes `bool` and `char` to `int`; if one side is then `double` the other becomes `double`; and if a signed and an unsigned value of the same size meet, the signed one becomes unsigned. My examples: `true + 2` is 3; `7 / 2.0` is 3.5 because the 7 becomes `7.0`; `-1 < 1u` is false because −1 becomes 4,294,967,295. The last one is the surprise of Task 4.

► Step 4: Ranges and the network total

`INT_MAX` is 2,147,483,647. With about Rs 19 per journey on average (my Task 10 data: 1899.95 paise), 3,000,000 journeys come to roughly 5,700,000,000 paise, and the program confirmed that this does not fit in an `int`. So `int` cannot hold one day's network collection, and Task 10 finds the exact journey at which it fails. A `long long` holds up to 9.22 × 10^18 paise, about 1.6 billion days of the network total, so it is enough with a large margin.

Alternative Concept and Complexity Analysis

If money were kept as `double` rupees, the solution would be: table `{10.0, 20.0, 30.0}`, surcharge as `fare * 1.2`, test `balance - fare >= 50.0`. It reads naturally, but every comparison of money can be hit by the rounding above.

Point	`long long` **paise**	`double` **rupees**
Time per addition (measured, -O2)	0.15 ns	0.59 ns
Space per value	8 bytes	8 bytes
Big-O	O(1) time, O(1) space	O(1) time, O(1) space
Exactness	Exact always	Error grows with n (table above)
Equality tests	Safe	Unsafe (`3 * 0.1 == 0.3` is false)

Table 13: Integer paise against double rupees.

The integer loop was also faster here, but I do not give all the credit to the type: the compiler could vectorise the integer sum and not the `double` sum (floating-point addition is not associative), so some of that gap is compiler behaviour. Either way the cost per addition is tiny against the risk of a wrong total.

Result of Task 3: a justified type for every value, measured evidence of one precision problem (drift of double totals) and one overflow problem (`int` at network size), both resolved by `long long` paise.

Task 4 – Control Implicit Conversions (4 Marks)

► Step 1: Where C++ converts without being told

Place	**Conversion**	**Example in the fare program**	**Risk**
Integer division	`int / int` stays integer, then widens to double when stored	`double c = a / b;` with 7 and 2 gives 3	Fraction lost before the double sees it
Integer to double	The int operand becomes double	`7 / 2.0` gives 3.5	Harmless, but hides where doubles enter money
Char promotion	`char` becomes `int` in arithmetic	`ch + 1` prints a number, not a letter	Output type changes silently
Int to bool	Any non-zero value becomes `true`	`bool flag = 5;`	A wrong value is hidden by the conversion
Signed with unsigned	The signed value becomes unsigned	`-1 < 1u` is false	Negative values turn into huge ones
Narrowing	Value cut to fit a smaller type	`int x = 3.7;` gives 3	Silent wrong fare

Table 14: Implicit conversion points and their risks.

► Step 2: Prediction written before running

I worked the program out on paper first. `7 / 2` is integer division, so `c` is 3 and prints as `3`; `d` is 3.5. `'A' + 2` is `'C'`, and `ch + 1` is an int (67 + 1 = 68). `bool flag = 5` becomes true, which prints as `1` and then, after `boolalpha`, as `true`. For the last line `s` converts to unsigned and becomes 4,294,967,295, which is not less than 1.

```
My predicted output (written before running):
3 3.5
C 68
1 true
s >= u
```

► Step 3: Actual output and explanation

Actual output (g++ -std=c++17 -Wall -Wextra t4_predict.cpp)

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning

```
t4_predict.cpp:13:21: warning: comparison of integer expressions of different signedness: ‘int’ and
    ‘unsigned int’ [-Wsign-compare]
```

Prediction and output are identical, so there was no difference to explain. The lines map to the conversions as follows:

Output line	Conversion that took place	Why
`3 3.5`	Integer division, then int to double	`a / b` is computed as int (3) before it is stored in a double. `static_cast<double>(a) / b` converts first, so the division is a double division.
`C 68`	char to int promotion	`ch` prints as a character; `ch + 1` is an int expression, so it prints 68.
`1 true`	int to bool	5 is non-zero so `flag` is true; `boolalpha` changes how it prints, not its value.
`s >= u`	signed to unsigned	`s` becomes 4294967295 before the comparison; the compiler's `-Wsign-compare` warning points at exactly this line.

Table 15: Each output line and the conversion behind it.

► Step 4: Narrowing and brace initialisation

Brace initialisation refuses narrowing conversions at compile time:

```
t4_narrow.cpp:3:11: error: narrowing conversion of ‘3.7000000000000002e+0’ from ‘double’ to ‘int’
    [-Wnarrowing]
t4_narrow.cpp:4:12: error: narrowing conversion of ‘5’ from ‘int’ to ‘bool’ [-Wnarrowing]
```

The same mistake written with `=` compiles and `-Wall -Wextra` stay silent. It is caught only if I add `-Wconversion`:

Output of t4_silent.cpp (no warning with -Wall -Wextra)

```
3 3600
```

Extra warnings with -Wconversion added

```
t4_silent.cpp:4:13: warning: conversion from ‘double’ to ‘int’ changes value from
    ‘3.7000000000000002e+0’ to ‘3’ [-Wfloat-conversion]
t4_silent.cpp:6:30: warning: conversion from ‘double’ to ‘int’ may change value [-Wfloat-conversion]
```

`static_cast` is needed when I want a conversion the compiler would not do on its own (`static_cast<Zone>(zoneNumber)` after the number has been range-checked) or would do unsafely (casting to `long long` before comparing signed and unsigned). The corrected version of the four lines is `t4_fix.cpp`:

Output of t4_fix.cpp (brace initialisation, explicit casts)

```
3.5
C 68
true
s < u
```

Note the difference in the third line: `bool flag{5 != 0}` says what I mean, and the fourth line now prints `s < u` because both sides are cast to `long long` first. That is the mathematically correct answer, which the original program did not give.

Alternative Concept and Complexity Analysis

If every conversion were written with `static_cast` and every variable brace-initialised, the solution is `t4_fix.cpp`. Compared with the implicit version:

Point	Implicit conversions	Explicit `static_cast` + braces
Generated code	—	Assembly for the same function written both ways is **identical**
Time per operation (measured, -O2)	1.24 ns	1.23 ns
Memory	No change	No change
Big-O	O(1)	O(1)
Errors caught at compile time	Few: only `-Wsign-compare`, `-Wconversion` if enabled	`int x{3.7}` and `bool f{5}` are hard errors
Errors found only at run time	Wrong fare after truncation, wrong comparison of −1 and 1u	Fewer, because every conversion is visible in the source

Table 16: Implicit against explicit conversions.

Result of Task 4: six conversion points found and demonstrated, two of them genuine surprises (integer division and signed/unsigned comparison), and corrected code that costs nothing at run time.

Task 5 – Write Correct Expressions with the Right Precedence (4 Marks)

► Step 1: The three expressions of the fare system

```
// peak check: minutes since midnight, so 08:00 and 17:00 are peak, 10:00 and 20:00 are not
const int t = hour * 60 + minute;
bool peak = (t >= PEAK_AM_START && t < PEAK_AM_END) || (t >= PEAK_PM_START && t < PEAK_PM_END);

// surcharge: integer paise, rounded half-up
f += pct(f, PEAK_SURCHARGE_PERCENT);          // pct(a, p) = (a * p + 50) / 100

// concession
f -= pct(f, concessionPercent(category));
```

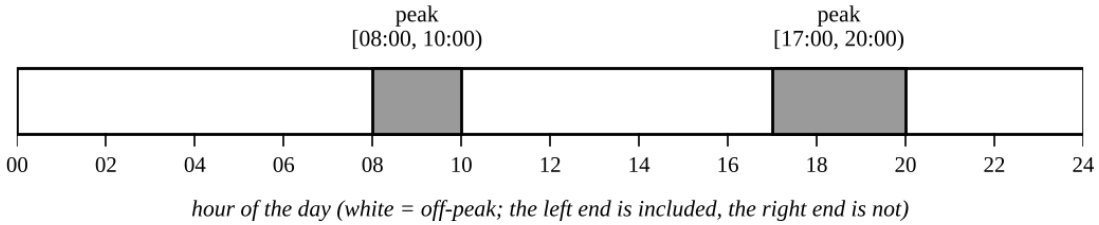


Figure 4: The two peak windows; each includes its first minute and excludes its last.

Precedence and associativity. In `hour * 60 + minute` the `*` is applied first. In the peak test the comparisons (`>=`, `<`) bind tighter than `&&`, and `&&` binds tighter than `||`. In `(a * p + 50) / 100` the brackets make the `+` happen before the `/`; without them `a * p + 50 / 100` would add 0 and round nothing. `*` and `/` have equal precedence and go left to right, so `a * p / 100` means `(a * p) / 100`, which is the right order for integers because the multiplication keeps the precision. The compound assignments `+=` and `-=` have the lowest precedence and go right to left. The conditional operator `?:` sits just above assignment, so `x > y ? x : y` needs no brackets by itself but does when it is part of a longer expression, as in the program below.

`(t >= 480 && t < 600) || (t >= 1020 && t < 1200)` and the same without brackets mean the same thing, because `&&` is applied before `||`. I keep the brackets: they show the two windows at a glance, and g++ suggests them (`-Wparentheses`) when `&&` and `||` are mixed.

► Step 2: x++ against ++x, and why i = i++ is a mistake

`x++` gives the old value and then adds 1; `++x` adds 1 and then gives the new value. I tested both, and the mistaken `i = i++`:

`Output of t5_inc.cpp`

```
i = i++  gives 5
k=5 m=7 j=7
```

Warning (g++ -std=c++17 -Wall -Wextra)

```
t5_inc.cpp:5:7: warning: operation on ‘i’ may be undefined [-Wsequence-point]
t5_inc.cpp:5:7: warning: operation on ‘i’ may be undefined [-Wsequence-point]
```

`j++` gave 5 (old value) and `++j` gave 7. In `i = i++` the old value 5 is written back after the increment, so the increment is lost and `i` stays 5. Before C++17 this was undefined behaviour; since C++17 the result is defined but the line is still pointless and the compiler still warns. What was meant is just `i++;`.

► Step 3: / and % with negative operands

Since C++11 integer division truncates toward zero and the remainder takes the sign of the left operand, so that `(a / b) * b + a % b == a` always holds. Hence `10 / 3 = 3`, `10 % 3 = 1`, `-10 / 3 = -3` and `-10 % 3 = -1`.

► Step 4: Prediction written before running

```
My predicted output (written before running):
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual output (g++ -std=c++17 -Wall -Wextra t5_predict.cpp; no warnings)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Prediction and output are identical. Operator by operator:

Line	Operators and the order they are applied	Result
`x + y * 2`, `(x + y) * 2`	`*` before `+`: 5 + (3·2) = 11. With brackets the `+` goes first: (5 + 3)·2 = 16	`11 16`
`int z = x++ + ++y;`	`x++` yields 5 and makes x = 6; `++y` makes y = 4 and yields 4; then `+`: z = 5 + 4. The two increments touch different variables, so the result is well defined. Then `cout` prints z, x, y	`9 6 4`
`(x > y ? x : y)`	Brackets, then `>`: 6 > 4 is true, so `?:` yields x	`6`
`x % 4 * 2`	`%` and `*` have the same precedence and go left to right: (6 % 4) · 2 = 2 · 2	`4`
`10 / 3`, `10 % 3`	Integer division truncates: 3; remainder 1	`3 1`
`-10 / 3`, `-10 % 3`	Unary minus first, then truncate toward zero: −3; the remainder takes the sign of the dividend: −1	`-3 -1`
`for (i = 0; i < 10; i += 3) k += i;`	i takes 0, 3, 6, 9 (the test `i < 10` stops it before 12); k = 0 + 3 + 6 + 9	`18`

Table 17: Operators involved in every line of the Task 5 program.

Alternative Concept and Complexity Analysis

If the peak test used a lookup table indexed by the hour, the solution would be:

```
bool peakTable[24];
for (int h = 0; h < 24; ++h)
    peakTable[h] = (h >= 8 && h < 10) || (h >= 17 && h < 20);   // filled once at start-up
...
bool peak = peakTable[hour];                                    // one array access per check
```

Point	Expression	Table `bool peak[24]`
Time per check (measured, -O2)	0.89 ns	0.29 ns
Comparisons per check	up to 4 comparisons and 2 logical operators	none: one array access

Point	Expression	Table `bool peak[24]`
Extra space	none	24 bytes
Big-O	O(1)	O(1) time, O(1) space (24 bytes)
Weak point	—	Works only while windows start and end on whole hours; a 17:30 start needs a minute table of 1440 entries. The peak rule is also written twice (the loop and the constants) and the two can drift apart

Table 18: Peak check by expression against lookup table.

The table is about three times faster per check in my loop, but both are far below anything a gate would notice, and the expression works on minutes with no change. I keep the expression.

Result of Task 5: correct, readable expressions for peak, surcharge and concession, with the 10:00 and 20:00 boundaries settled by `<` on a minute scale, and a predicted output that matched on all five lines.

Task 6 – Design the Control Structures (4 Marks)

► Step 1: Choosing the structure

Decision	Structure chosen	Reason
Base fare from the zone	`switch` on `Zone`	One variable against a few fixed values; with an `enum class` g++ warns (`-Wswitch`) if a value is not handled
Peak or off-peak; concession or not	`if` / `else`	The conditions are ranges and combined tests, which a `switch` cannot express
Choosing between two values inside an expression	Conditional operator `?:`	Short two-way choice that returns a value, such as `"allowed" : "denied"`; clear only when it fits one line
Allow or deny the journey	`if` / `else` on `canTravel()`	Two outcomes with different statements

Table 19: Which control structure for which decision.

Loop	Sketch	Judgement
`for`	`for (size_t i = 0; i < n; ++i) trips[i]`	Works, but the index can run past the end if `n` is wrong
range-based `for`	`for (const Trip& t : trips)`	Best for a stored list: visits each journey once, no index, no copy
`while`	`while (std::cin >> zone >> h >> m)`	Best when the number of journeys is not known and comes from input (Task 8)

Table 20: Looping over a day's journeys.

break and continue. `break` ends a `switch` case (without it execution falls into the next case) or leaves a loop. `continue` skips an invalid journey and moves to the next one, so one bad record does not stop the day; the gate program of Task 8 uses it. **No case matches:** a `switch` does nothing, so in my code `base` stays at its starting value 0 and the passenger would travel free. That is dangerous, and it is exactly what happens in Task 12 when a fourth zone is added without a new case.

► Step 2: Pseudocode before code

```
ComputeFare(z, peak)
  switch z
    case Inner  : base <- BASE_PAISE[0]
    case Middle : base <- BASE_PAISE[1]
    case Outer  : base <- BASE_PAISE[2]
  IF peak THEN base <- base + (base * PEAK_SURCHARGE_PERCENT + 50) / 100   // integer paise only
  RETURN base

CanTravel(balance, f)
  RETURN balance - f >= MIN_BALANCE_PAISE

ProcessDay(journeys, balance)
  FOR EACH j IN journeys
    IF j is invalid THEN CONTINUE
    f <- ComputeFare(j.zone, j.peak)
    IF CanTravel(balance, f) THEN balance <- balance - f
    ELSE report "Denied"
```

► Step 3: Complete the Code

I copied the given program into `t6_complete.cpp` and replaced only the two TODO bodies:

```
long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {                                                    // TODO 1
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += (base * PEAK_SURCHARGE_PERCENT + 50) / 100;   // integer paise, half-up
    return base;
}
bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;                        // TODO 2
}
```

Output (no warnings with -Wall -Wextra)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

This is the expected output. The trace: Inner off-peak costs 1000, balance 10000 → 9000. Outer peak costs 3000 + 600 = 3600, balance 9000 → 5400. Middle off-peak costs 2000 and would leave 3400, below 5000, so it is denied and the balance stays 5400. (The given `main` prints `balance / 100`, which drops the paise; Task 8 prints both parts.) I used the half-up form `(base * 20 + 50) / 100` so the result agrees with `fare_core.h`; for these fares it gives the same 600 as `base * 20 / 100`.

Journeys at exactly 10 AM and 8 PM. The given `fare(Zone, bool)` receives the peak flag from outside, so the boundary rule lives in `isPeak()`. `t6_boundary.cpp` tests the edges of both windows:

```
07:59          off-peak  Inner fare Rs 10.00
08:00          peak      Inner fare Rs 12.00
09:59          peak      Inner fare Rs 12.00
10:00 (10 AM)  off-peak  Inner fare Rs 10.00
16:59          off-peak  Inner fare Rs 10.00
17:00          peak      Inner fare Rs 12.00
19:59          peak      Inner fare Rs 12.00
20:00 (8 PM)   off-peak  Inner fare Rs 10.00
```

10:00 (10 AM) and 20:00 (8 PM) are off-peak, and 08:00 and 17:00 are peak. The old inconsistency is gone because both ends use < on the same minute scale.

Alternative Concept and Complexity Analysis

If the base fare came from an array indexed by the zone, the solution would be:

```
constexpr long long FARE[] = {1000, 2000, 3000};
long long fare(Zone z, bool peak)
{
    long long base = FARE[static_cast<int>(z)];       // no switch
    if (peak) base += (base * 20 + 50) / 100;
    return base;
}
```

Point	`switch`	**Array** `FARE[]`
Time per lookup (measured, -O2)	0.46 ns	0.31 ns
Big-O	O(1); the compiler may build a jump table	O(1)
Space	Values sit in instructions	3 × 8 = 24 bytes, O(z) for z zones
Fourth zone added	Add an enumerator and a `case`. If the `case` is forgotten the code compiles with a `-Wswitch` warning and the fare is 0	Add one element. If it is forgotten, `FARE[3]` reads past the array and the compiler says nothing; a `static_assert` on the array size is needed
Scales to many zones	Gets long	Stays one line

Table 21: Switch against array for the base fare.

The array is shorter and scales better, so Task 12 uses it with a `static_assert` on its size. The switch is safer today only because g++ warns about a missing enumerator; a warning is easy to miss in a long build.

Result of Task 6: control structures chosen with reasons, the completed program matching the expected output, and the 10 AM / 8 PM edge cases settled.

Task 7 – From Procedural to Object-Oriented (4 Marks)

► Step 1: The procedural version

`t7_proc.cpp` keeps `balance` as a global and uses free functions. Nothing forces the caller to ask `canTravel()` before `deduct()`.

```
long long balance = 10000;                    // global: every function can change it
const long long BASE[] = {1000, 2000, 3000};
const long long MIN_BAL = 5000;
...
bool canTravel(long long f) { return balance - f >= MIN_BAL; }
void deduct(long long f) { balance -= f; }    // nothing forces the caller to ask canTravel() first
```

Output (g++ -std=c++17 -Wall -Wextra, no warnings)

```
paid 1000  balance 9000
paid 3600  balance 5400
denied   balance 5400
after a careless deduct(9999): -4599
```

The first three lines are correct. The last shows the weakness: `deduct(9999)` is a legal call, so the balance becomes −4599 and the original negative-balance bug is back.

► Step 2: The object-oriented version

```
class SmartCard {
    long long balance_;
    static constexpr long long MIN_BAL = 5000;
public:
    explicit SmartCard(long long b) : balance_(b) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return balance_ - f >= MIN_BAL; }
    bool deduct(long long f)
    {
        if (!canTravel(f)) return false;      // the rule lives here, once
        balance_ -= f;
        return true;
    }
};
```

Output (no warnings)

```
paid 1000  balance 9000
paid 3600  balance 5400
denied   balance 5400
careless deduct(9999) accepted? false, balance 5400
```

The first three lines match the procedural version. The careless `deduct(9999)` is refused and the balance stays 5400. Because `balance_` is private, the careless assignment cannot even be written:

g++ -std=c++17 t7_err_private.cpp

```
t7_err_private.cpp:5:10: error: ‘long long int SmartCard::balance_’ is private within this context
```

► Step 3: How the class prevents a negative balance

`balance_` can change only inside `deduct()` and `recharge()`. `deduct()` calls `canTravel()` first and changes nothing when `balance_ - f < 5000`; `recharge()` refuses amounts of zero or less; the constructor clamps a negative opening value to 0. Every public function keeps the rule true, so no sequence of calls can reach a negative balance. The balance can fall below Rs 50 only if the card is created below Rs 50, which is a data-entry matter and not a gate bug.

Point	Procedural	Object-oriented
Where the balance lives	Global variable, visible to every function	Private member of `SmartCard`
Who enforces Rs 50	Every caller must call `canTravel()` first	`deduct()` enforces it itself
Careless `deduct(9999)`	Balance becomes −4599	Refused; balance stays 5400
Change the minimum balance	One constant, but every function that uses `balance` must be reviewed	One constant inside `SmartCard`; only 4 member functions can use it
Amount of code	Slightly shorter	A few more lines (class, constructor)
Time and space per journey	O(1), 8 bytes	O(1), 8 bytes (Task 1: 1.72 ns against 2.41 ns)

Table 22: Procedural and object-oriented versions compared.

Easier to change: the object-oriented version. In both programs 5000 appears once, but in the procedural one any new function can call `deduct()` directly and skip the rule. In the class the rule sits where the data is.

Alternative Concept and Complexity Analysis

If the balance were a public data member, the solution would be a plain struct:

Output of t7_public.cpp (struct SmartCard { long long balance; }) – compiles with no warning

```
balance after careless -=  : 1
balance after assignment  : -500
```

`card.balance -= 9999;` leaves 1, below Rs 50, and `card.balance = -500;` makes it negative. Neither is refused. Time and space do not change (O(1), 8 bytes), but any part of the program can now break R1 (Rs 50 minimum), the non-negative balance and the positive-recharge rule (R6). The cost of the public member is not speed; it is that these rules can no longer be proved by reading one class.

Result of Task 7: two working versions, one reproduced bug (−4599) that the class makes impossible, and a compile error that shows the protection is enforced by the language.

Task 8 – Read and Print Data Safely (4 Marks)

► Step 1: What goes wrong with unchecked cin

If the operator types letters where a number is expected, `cin >> n` puts the stream into a fail state, stores 0 and leaves the letters in the buffer. Every later read then does nothing. Recovery takes three steps: test the read (`if (!(cin >> n))`), call `cin.clear()` to leave the fail state, then `cin.ignore(max, '\n')` to throw the bad line away.

► Step 2: The gate program

```
int main()
{
    Passenger p("Saran", Category::Regular, 10000);              // Rs 100.00 on the card
    int n = 0;
    std::cout << "How many journeys? ";
    while (!(std::cin >> n) || n < 0) {
        if (std::cin.eof()) return 1;
        std::cout << "\n  that is not a valid count, try again\nHow many journeys? ";
        dropLine();
    }
    std::cout << "\n\n" << std::left << std::setw(4) << "No" << std::setw(7) << "Zone" << std::setw(8)
        << "Time"
              << std::right << std::setw(10) << "Fare(Rs)" << std::setw(14) << "Balance(Rs)" << '\n'
              << std::string(43, '-') << '\n';
    for (int i = 1; i <= n; ++i) {
        int zi, h, m, ci;
        if (!(std::cin >> zi >> h >> m >> ci)) {                  // letters instead of numbers, or
            end of input
            if (std::cin.eof()) { std::cout << "input ended early\n"; break; }
            std::cout << std::left << std::setw(4) << i << "rejected: not a number\n";
            dropLine();
            continue;
        }
        Zone z;
        Category c;
        if (!toZone(zi, z) || !validTime(h, m) || !toCategory(ci, c)) {
            std::cout << std::left << std::setw(4) << i << "rejected: value out of range\n";
            continue;
        }
        const long long f = FareTable::fare(z, h, m, c);
        std::cout << std::left << std::setw(4) << i << std::setw(7) << zi << std::setfill('0') <<
            std::right
                  << std::setw(2) << h << ':' << std::setw(2) << m << std::setfill(' ') << "   ";
        if (p.card().deduct(f))
            std::cout << std::setw(10) << money(f) << std::setw(14) << money(p.card().balance()) <<
                '\n';
        else
            std::cout << std::setw(10) << "DENIED" << std::setw(14) << money(p.card().balance()) <<
                '\n';
    }
    ...
```

Each journey is read as `zone hour minute category`. `<iomanip>` does the layout: `setw` sets a column width, `setfill('0')` pads the time (09:30), and the money comes from `money()`, which prints integer paise as `rs.paise`, so no floating point touches the amount. A `fixed` / `setprecision(2)` version of the closing balance is printed once to show that option; it is used for display only.

► Step 3: Runs with invalid input

Input file `t8_a.in` has letters in the hour of journey 2 (`2 abc 10 0`) and an unknown zone in journey 4 (`3 10 0 0`):

t8_a.in

```
5
0 9 30 0
2 abc 10 0
1 18 5 0
3 10 0 0
2 20 0 0
```

Output of ./t8_gate < t8_a.in

```
How many journeys?

No  Zone   Time      Fare(Rs)   Balance(Rs)
-----------------------------------------
1   0      09:30        12.00         88.00
2   rejected: not a number
3   1      18:05        24.00         64.00
4   rejected: value out of range
5   2      20:00       DENIED         64.00

closing balance, fixed + setprecision(2): 64.00
```

Journeys 2 and 4 are reported and skipped, and journey 5 is still processed. Journey 5 (Outer at 20:00) is off-peak, Rs 30.00, but the balance would drop to Rs 34.00 (below Rs 50), so it is denied. Journey 3 (18:05) is peak: Rs 20 + 20% = Rs 24.00. For a bad count (`x`, then `-3`, then `2`):

Output of ./t8_gate < t8_b.in

```
How many journeys?
  that is not a valid count, try again
How many journeys?
  that is not a valid count, try again
How many journeys?

No  Zone   Time      Fare(Rs)   Balance(Rs)
-----------------------------------------
1   0      08:00        12.00         88.00
2   2      10:00        30.00         58.00

closing balance, fixed + setprecision(2): 58.00
```

The letters and the negative count are rejected and the question repeats; then two journeys run. 08:00 is peak (Rs 12.00) and 10:00 is off-peak (Rs 30.00).

A weakness of `cin >>` showed up when I tried a record with a missing field. File `t8_c.in` has the line `1 9` with the minute and category missing:

Output of ./t8_gate < t8_c.in

```
How many journeys?

No  Zone   Time      Fare(Rs)   Balance(Rs)
-----------------------------------------
1   rejected: value out of range
2   rejected: value out of range
input ended early

closing balance, fixed + setprecision(2): 100.00
```

The reads ran on into the next line: `1 9 0 10` became zone 1, hour 9, minute 0, category 10, and the next record was then read out of step. Records were treated as one stream of numbers, not as lines. Nothing was charged wrongly, but the two good records on lines 3 and 4 were swallowed (the run ended early). This is the main reason for the alternative below.

Alternative Concept and Complexity Analysis

If each input line were read with `getline()` and converted with `stoi()` inside `try/catch`, the solution is `t8_getline.cpp`:

```
static bool toInt(const std::string& s, int& out)
{
    try {
        size_t pos = 0;
        const int v = std::stoi(s, &pos);
        if (pos != s.size()) return false;                          // "7x" is not a number
        out = v;
        return true;
    } catch (const std::exception&) {                               // invalid_argument or
        out_of_range
        return false;
    }
}
```

Output of `./t8_getline < t8_a.in`

```
count 5
line 2: paid Rs 12.00  balance Rs 88.00
line 3: not a number
line 4: paid Rs 24.00  balance Rs 64.00
line 5: value out of range
line 6: DENIED  balance Rs 64.00
```

Output of `./t8_getline < t8_c.in`

```
count 3
line 2: need exactly 4 fields
line 3: paid Rs 10.00  balance Rs 90.00
line 4: paid Rs 30.00  balance Rs 60.00
```

Here the bad line is rejected as “need exactly 4 fields” and the next line is processed correctly (lines 3 and 4 are paid). `stoi` throws `invalid_argument` for `abc` and `out_of_range` for numbers that are too large, and `7x` would be accepted as 7 unless the `pos` check is added, which I did.

Point	`cin >>`	`getline` + `stoi` + `try/catch`
Time for a line of length L	O(L)	O(L)
Measured, 1,000,000 lines read from memory	119 ms	82 ms
Extra space	No per-line storage	A string of L bytes: O(L)
Recovery after bad input	`clear()` and `ignore()` by hand; a short line can pull in the next line	Each line is independent; only that line is rejected
Error reporting	`fail()` says only “something was wrong”	Exceptions separate not-a-number from too-large; I can report the line number

Table 23: Direct cin extraction against line-based reading.

The timings come from an in-memory `istringstream`, not a keyboard, so they show only the parsing cost. For interactive typing speed is irrelevant; for a file of records I would use `getline` because of the line independence shown above. For this assignment I kept `cin >>` in the main gate program because Task 8 asks for it, and I report its weakness honestly.

Result of Task 8: safe input with demonstrated recovery from letters, bad ranges and a bad count, a neat report, and a documented case where `cin >>` swallows two good records that the line-based version keeps.

Task 9 – Implement and Test the Fare Logic (4 Marks)

The fare logic is in `fare_core.h` (Tasks 2, 5 and 6). `t9_tests.cpp` has a table of cases; **every expected value was worked out by hand before the program was run**, for example Outer at 08:30 is 3000 + 20% of 3000 = 3600, and a student in the Inner zone at 09:00 is 1000 − 250 = 750, then 750 + 150 = 900. The balance, invalid-input and five-journey checks follow the table.

► Step 1: The test table and its result

No.	Case	Expected	Actual	Result
1	Inner off-peak 12:00	1000	1000	PASS
2	Middle off-peak 12:00	2000	2000	PASS
3	Outer off-peak 12:00	3000	3000	PASS
4	Inner peak 09:00	1200	1200	PASS
5	Middle peak 18:00	2400	2400	PASS
6	Outer peak 08:30	3600	3600	PASS
7	boundary 07:59 (just before)	1000	1000	PASS
8	boundary 08:00 (first peak minute)	1200	1200	PASS
9	boundary 09:59 (last peak minute)	1200	1200	PASS
10	boundary 10:00 (off-peak)	1000	1000	PASS
11	boundary 17:00 (first peak minute)	1200	1200	PASS
12	boundary 19:59 (last peak minute)	1200	1200	PASS
13	boundary 20:00 (off-peak)	1000	1000	PASS
14	student Outer off-peak	2250	2250	PASS
15	student Inner peak	900	900	PASS
16	senior Middle off-peak	1000	1000	PASS
17	senior Outer peak	1800	1800	PASS
18	balance 6000, fare 1000 (leaves 5000)	allowed	allowed	PASS
19	balance 5999, fare 1000 (leaves 4999)	denied	denied	PASS
20	balance 6000, fare 1001 (leaves 4999)	denied	denied	PASS
21	unknown zone 7 rejected	rejected	rejected	PASS
22	zone 2 accepted	accepted	accepted	PASS
23	negative recharge -200 rejected	rejected	rejected	PASS
24	zero recharge rejected	rejected	rejected	PASS
25	balance unchanged after bad recharges	8000	8000	PASS
26	hour typed as 'ab' rejected	rejected	rejected	PASS
27	hour typed as '9x' rejected	rejected	rejected	PASS
28	hour 25 rejected	rejected	rejected	PASS
29	five journeys, total by hand	8900	8900	PASS

Table 24: Test table with expected and actual results for every case (output of t9_tests.cpp; fares and balances in paise).

Result: **29 of 29 checks passed**. The rows cover normal journeys in each zone (1–6), the boundaries of both peak windows (7–13), concessions (14–17), the Rs 50 edge (18–20) and invalid input (21–28). Row 29 is the five-journey total.

Five journeys by hand. Inner 07:59 regular = 1000; Middle 08:00 regular, peak = 2000 + 400 = 2400; Outer 17:30 regular, peak = 3000 + 600 = 3600; Inner 09:00 student, peak = 750 + 150 = 900; Middle 20:00 senior, off-peak = 1000. Total = 1000 + 2400 + 3600 + 900 + 1000 = **8,900 paise (Rs 89.00)**, which the program matched.

► Step 2: Do the tests actually catch a bug?

A table that always passes proves little. I built the same tests against a deliberately wrong `isPeak()` that uses `<=` at the morning end (the original 10 AM defect), by compiling with `-DMUTANT_PEAK_INCLUSIVE`:

```
10  boundary 10:00 (off-peak)              1000      1200      FAIL
28 of 29 checks passed
```

Exactly one test fails, and it is the 10:00 boundary row. Without the boundary rows this bug would have passed all the ordinary cases, which is why the boundaries are in the table.

Alternative Concept and Complexity Analysis

The fare rows (1–17) are table-driven: one `Case` struct and one loop. The alternative is a hand-written check for each case (`if (fare(...) != 1200) std::cout << "FAIL"`). Rows 18–29 use a small `report()` function for mixed result types.

Point	Table-driven	Separate hand-written checks
Time for t tests	O(t)	O(t)
Measured for t = 1,000,000 cases	5.5 ms	not built: it would need a million statements
Space	O(t): 24 bytes per row, so about 24 MB for 1,000,000 rows	O(1) data, but O(t) program text
Adding a boundary case	one new row of data	one new statement, easy to copy wrongly
Reading the failures	one report format for all	each check formats its own message

Table 25: Table of test cases against separate checks.

Result of Task 9: 29 checks, all passing, with the 10:00 boundary proved able to fail by a mutation test and a five-journey total matched against a hand calculation.

Task 10 – Compare Expectations with Execution (4 Marks)

► Step 1: The experiment and what I expected

`t10_totals.cpp` makes 3,000,000 journeys with a fixed seed (random zone, an hour from 05 to 22, and a concession for 30% of passengers: 20% students, 10% seniors). It adds the fares four ways: exact `long long` paise; `double` rupees using the hurried version's formula (multipliers `0.75`, `0.5`, `1.2`); `float` rupees; and an `int` total watched with `__builtin_add_overflow`. I expected `long long` and `double` to agree for these fares, `float` to lose accuracy once the total gets large, and `int` to overflow at network size.

Journeys	long long (paise)	double → paise	Difference	float total (Rs)	float error (Rs)	int total
20,000	38,036,450	38,036,450	0	380,364	< 1	38,036,450 (fits)
500,000	950,628,800	950,628,800	0	9,506,275	-13	950,628,800 (fits)
3,000,000	5,699,862,900	5,699,862,900	0	57,146,416	+147787	overflowed (wraps to 1,404,895,604)

Table 26: Totals compared for 20,000, 500,000 and 3,000,000 journeys (measured, t10_totals.cpp).

► Step 2: Does int overflow for the network total?

Yes. The exact total of the 3,000,000 journeys is 5,699,862,900 paise (Rs 56,998,629.00), far above `INT_MAX` = 2,147,483,647. The first journey at which the `int` total overflows is journey **1,130,039**, so one gate line with 500,000 journeys a day is still safe but the whole network is not. The wrapped value in the table is the exact total minus 2^32; strictly, signed overflow is undefined behaviour in C++, so I watched it with `__builtin_add_overflow` and showed the wrapped value from unsigned arithmetic, which is defined.

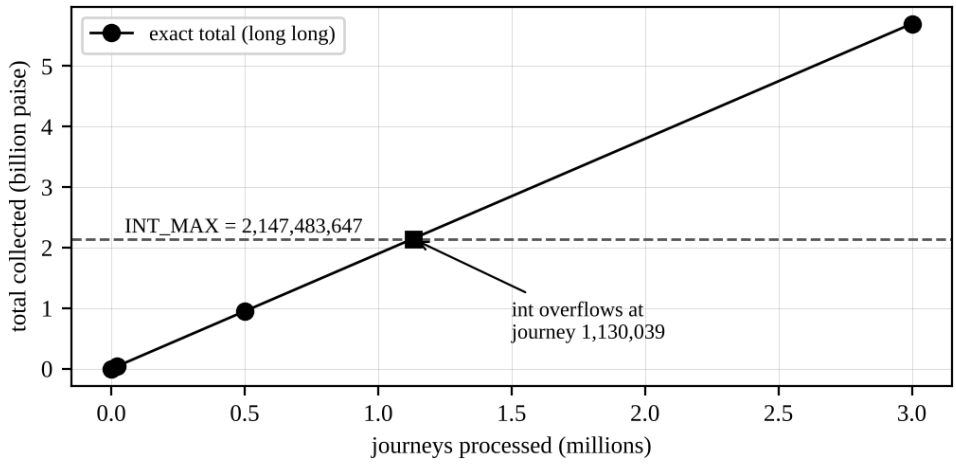


Figure 5: Running network total against the largest value an int can hold.

► Step 3: Why the totals differ, or do not

- **double agreed exactly** (difference 0 at all three sizes). Every fare here is a multiple of 50 paise (Rs 7.50, 9.00 and so on) and every partial sum stays far below 2^53, so no rounding happened. The hurried formula also gave the same paise as `fare()` for every one of the 3,000,000 journeys (0 mismatches). This is luck of the rates, not safety: Task 3 shows `double` drifting for Rs 10.80.
- **float lost Rs 147,787 at 3,000,000 journeys.** A `float` has about 7 decimal digits. Once the running total is large, adding a Rs 12–36 fare is rounded to the nearest representable value, and many additions round the same way. At 20,000 journeys the error was under Rs 1 and at 500,000 it was Rs −13; the damage grows quickly.
- **int wrapped.** The type is too small; the cure is a wider type, not better arithmetic.

Out of the 3,000,000 journeys, 833,365 (27.8%) were peak journeys in this synthetic sample; the average fare was 1899.95 paise.

► Step 4: Compiler warnings with -Wall -Wextra

I compiled all 21 programs with `-Wall -Wextra`. Eighteen compiled with no warning. These are the warnings I received, and one extra set that appears only with `-Wconversion`:

Program	Warning	What it meant	Action
t4_predict.cpp	`-Wsign-compare`: comparison of `int` and `unsigned int`	−1 is converted to 4294967295 before the comparison	Cast both sides to `long long` (t4_fix.cpp)
t5_inc.cpp (twice)	`-Wsequence-point`: operation on `i` may be undefined	`i = i++` modifies and reads `i` in one expression	Write `i++;`
t12_old.cpp	`-Wswitch`: enumeration value `Airport` not handled	The switch has no case for the new zone, so the fare is 0	Replace the switch with a table and `static_assert` (Task 12)
t4_silent.cpp (only with `-Wconversion`)	`-Wfloat-conversion`: double to int changes value	`int x = 3.7;` silently becomes 3	Brace-initialise or cast explicitly

Table 27: Warnings produced by the build and what each one pointed at.

Each of the first three pointed at a real defect, not at style. Two of them (`-Wswitch` and `-Wsign-compare`) are only warnings, so the program still builds and runs; I recommend `-Werror` for the gate software so that they stop the build.

Alternative Concept and Complexity Analysis

If the network total were kept in an `int`, a `long long` or a `double`, the result at 3,000,000 journeys would be:

Accumulator	Result at 3,000,000 journeys	Failure mode	sizeof	Time per addition (measured)
`int` (paise)	1,404,895,604 (wrapped; undefined behaviour in C++)	Overflow, from journey 1,130,039	4 bytes	0.08 ns (unsigned)
`long long` (paise)	5,699,862,900 – exact	None until about 9.2 × 10^18 paise	8 bytes	0.15 ns
`double` (rupees)	Rs 56,998,629.00 – correct for these fares	Loss of precision for fares with decimals (Task 3); exact integers only up to 2^53	8 bytes	0.59 ns
`float` (rupees)	Rs 57,146,416 – wrong by Rs 147,787	Loss of precision above 2^24 (about 16.7 million)	4 bytes	0.61 ns

Table 28: What each accumulator type does at network size.

The two failures are different in kind. **Overflow** means the value does not fit: the answer is wrong but looks precise. **Loss of precision** means the value fits but is not exact: the answer is close but wrong in the last digits. Every addition is O(1) with 4 or 8 bytes per variable. The `long long` loop is as fast as the `int` loop within a fraction of a nanosecond and is exact, so it is the right choice. (The `int` and `long long` loops were probably vectorised by the compiler, which flatters them against `double`; the conclusion does not depend on it.)

Result of Task 10: all four predictions held, plus two things I did not predict: the `double` result is correct only because of the chosen rates, and `float` is wrong by Rs 147,787 at network size.

Task 11 – Recommend and Defend the Final Design (4 Marks)

► Step 1: Recommended design and the evidence for it

Element	Recommendation	Evidence from my experiments
Money	`long long` paise everywhere; rupees only when printing	Task 3: `double` drifted by about Rs 0.019 at n = 10^7; Task 10: `int` wrapped from journey 1,130,039 and `float` was wrong by Rs 147,787; `long long` exact in every run
Constants	`constexpr` in one header: fares, percentages, minimum balance, peak limits	Task 2: macro `500 + 500` gave 1500 instead of 2000; one grep hit for the fare line
Zones and categories	`enum class`	Task 2: plain enum allowed `OOuter + 5`; enum class refused it
Peak rule	`isPeak()` on minutes since midnight, half-open intervals	Task 6 and 9: 10:00 and 20:00 off-peak, 08:00 and 17:00 peak; the mutation test failed exactly on 10:00
SmartCard	Private balance; `canTravel()`, `deduct()` with the check inside	Task 7: careless `deduct(9999)` gave −4599 in the procedural version and was refused by the class; the speeds are of the same order
Fare function	Integer percentage with half-up rounding	Task 9: 29 of 29 checks; Task 10: 0 mismatches in 3,000,000 fares
Input	`cin` with `fail()` / `clear()` / `ignore()` for numbers; `getline` for files of records	Task 8: bad input skipped and processing continued; `cin >>` swallowed two good records after a short line
Compiler flags	`-std=c++17 -Wall -Wextra`, and `-Werror` for release	Task 10: three real defects were flagged by warnings

Table 29: Recommended types, constants and classes, each with its evidence.

► Step 2: How a fare revision is carried out

1 Edit the one line in the constants block of `fare_core.h` (for example `{1000, 2000, 3000}` → `{1100, 2200, 3300}`).

2 Rebuild with `-Wall -Wextra`. On my machine one program takes about 0.27 s to compile, and no other file contains a fare.

3 Update the expected column of the test table (Task 9) and run it. The boundary rows already exist.

4 Deploy the new executable to the gates.

► Step 3: Limitations

- Assumptions A2 (25% / 50% concessions) and the half-up rounding are mine and must be confirmed with the metro authority.
- A fare change needs a rebuild and a redeployment (see the alternative below).
- Two gates updating one card at the same moment are not handled: a real system needs a database transaction around `deduct()`.
- Gate time is trusted as given; no clock or time-zone handling is included.
- The three-zone design has no weekend rule and no pass; Task 12 extends it.

► Step 4: Is the design correct for the whole network?

Yes, on the evidence. The exact 3,000,000-journey total (5,699,862,900 paise) fits in a `long long` with a margin of about 1.6 billion days. The fare function took 1.93 ns per journey in my timing, so 3,000,000 journeys cost about 6 ms of fare computation; the real limit at the gate will be card reading and the database, not this arithmetic. Memory is 8 bytes per card. What I have not tested is concurrent access from many gates; that is outside this assignment.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, the solution is `t11_config.cpp`:

```
std::vector<long long> loadFares(const std::string& path)
{
    std::ifstream in(path);
    std::vector<long long> v;
    long long x;
    while (in >> x) {
        if (x <= 0) throw std::runtime_error("fare must be positive");   // a bad file must be
            caught at run time
        v.push_back(x);
    }
    if (v.empty()) throw std::runtime_error("no fares found in " + path);
    return v;
}
```

Output of `t11_config.cpp`, with `fares.cfg` holding 1000, 2000, 3000, 6000

```
zone 0: 1000 paise
zone 1: 2000 paise
zone 2: 3000 paise
zone 3: 6000 paise
caught: no fares found in missing.cfg
```

Point	**Compiled in (`constexpr`)**	**Read from `fares.cfg`**
Start-up cost	none	1.29 µs to open the file and read 4 fares: O(z) for z fares
Space	24 bytes for 3 zones (static array)	z × 8 bytes in a `std::vector` (32 bytes for 4 zones)
Cost of a fare revision	recompile (about 0.27 s here) and redeploy the executable to every gate	edit one file and restart the gate
Safety	Compiler checks the type and the array size	A missing, empty or wrong file is found only at run time; my loader rejects an empty file and non-positive fares
Per-journey cost	O(1)	O(1) – the same lookup

Table 30: Compiled-in fare table against a configuration file.

The 1.3 µs start-up is negligible. The real comparison is operational: redeploying an executable to hundreds of gates takes far longer than the 0.27 s compile, so a configuration file wins if fares change often. It loses on safety, because the compiler can no longer check it. If I had to choose for a real metro I would use the file, but with the validation in `loadFares()`, a checksum, and the same test table run against the file at start-up. For this assignment, where fares are fixed by the scenario, the compiled-in table is what I recommend.

Result of Task 11: a recommendation in which every choice points to a measurement, an honest list of limits, and a clear statement of when the alternative would be better.

Day-4 Design Challenge

Task 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement. A fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones. My readings of the open points are assumptions A7 and A8 (Table 2). I chose to **modify** the design: keep what was correct and tested, and change only what the new rules touch.

► A. Impact analysis

Item in the original design	Affected?	How
`BASE_PAISE[]`	Yes	A fourth element 6000 (Airport)
`PEAK_SURCHARGE_PERCENT`, concession percentages, `MIN_BALANCE_PAISE`	No	Values unchanged
New constant `WEEKEND_DISCOUNT_PERCENT` = 10	New	A new rule needs a new named constant
`enum class Zone`	Yes	New enumerator `Airport` and a `Count` to size the table
`switch` in `fare()`	Yes	No case for Airport: it compiles with a warning and the fare is 0
`fare()` inputs	Yes	Only zone and peak (or hour and minute) go in; the day of the week and the pass cannot be expressed
`isPeak()`	No	Unchanged, but must not be applied on a weekend
Surcharge expression	Yes	Must become conditional on “not a weekend”
Concession expression	No	Unchanged (still applied first)
`SmartCard`	Partly	Balance logic is unchanged, but a covered journey must not be refused for a low balance
`Journey`	Yes	Needs the day, year and month
Pass	New	A new concept: two zones, a month, a coverage test
Input routine (Task 8)	Yes	Zone range 0–3; the day and date must be read and validated

Table 31: What the new requirements touch in the original design.

► B. Evaluation of the original design

I tested whether the original switch and `SmartCard` can absorb the three changes, by adding only the new enumerator to the Task 6/7 code (`t12_old.cpp`):

`Compiler message`

```
t12_old.cpp:8:12: warning: enumeration value ‘Airport’ not handled in switch [-Wswitch]
```

`Output of t12_old.cpp`

```
Example 1  Airport off-peak fare: 0 paise (should be 6000)
Example 2  Outer, Saturday 09:00, student: old code gives 3600 paise (new rules: 2025)
Example 3  Middle journey with a pass, balance Rs 10.00: refused when asked to deduct 0, refused when
    asked to deduct the normal fare
```

- **Example 1, fourth zone.** The code compiles with one warning and the Airport journey costs **0 paise instead of Rs 60.00**: a silent revenue loss. The switch is safe only if every switch in the program is updated by hand.

- **Example 2, weekend discount.** Outer, Saturday 09:00, student. By hand: 3000 − 25% = 2250; weekend 10% off → 2250 − 225 = **2025 paise (Rs 20.25)**, and no surcharge. The original `fare(Zone, bool peak)` cannot receive the day or the concession, and with `peak = true` it returns 3600, which is Rs 15.75 too much.
- **Example 3, pass.** A card with an Inner + Middle pass and Rs 10.00. A Middle journey must be free, but the original `deduct` tests 1000 − f ≥ 5000 and refuses even a 0 fare. The class has no place for a pass.

Requirement	Can the original absorb it?	Why
Fourth zone	Partly	Needs one enumerator and one case, but a forgotten case fails silently (Example 1)
Weekend discount	No	The function has no day input and the surcharge is unconditional (Example 2)
Monthly pass	No	`SmartCard` cannot represent a pass and always applies the Rs 50 test (Example 3)

Table 32: Can the original design absorb the changes?

► C. Adaptation: the revised design

I kept integer paise, constants in one place, the `pct()` rounding, the Rs 50 rule and the private balance, because every test of Task 9 still depends on them. I changed four things:

- A fare table `BASE_PAISE[ZONE_COUNT]` with `Zone::Count` replaces the switch; a new zone is one enumerator and one number, and a `static_assert` checks the size.
- Each pricing rule is a small class derived from `Adjustment` (`ConcessionRule`, `WeekendRule`, `PeakRule`). `FareEngine` applies them in order. The weekend rule and the peak rule exclude each other in one place.
- A `Journey` struct carries zone, time, day, category, year and month.
- A `Pass` class holds two zones and a month; `SmartCard::travel()` asks the pass first and deducts only if it does not cover the journey.

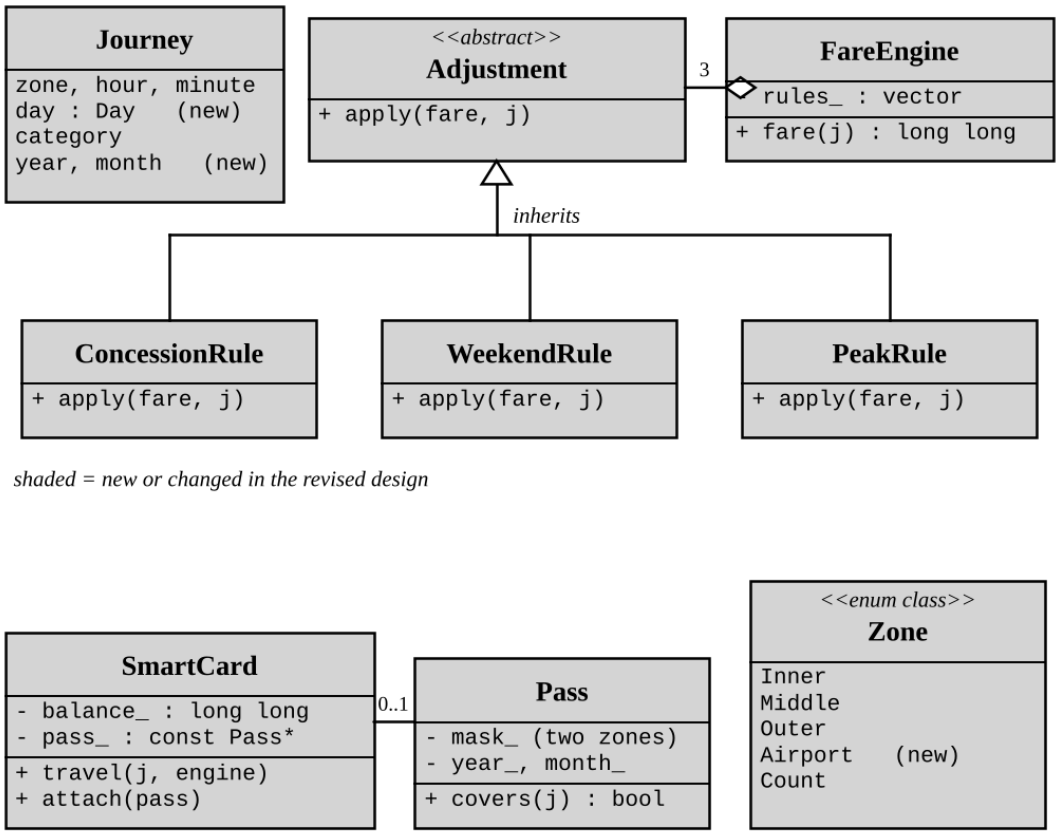


Figure 6: Revised class design for Task 12 (shaded boxes are new or changed).

fare_v2.h (extract): the rule classes and the engine

```
class Adjustment {
public:
    virtual ~Adjustment() = default;
    virtual long long apply(long long fare, const Journey& j) const = 0;
};
class ConcessionRule : public Adjustment {
public:
    long long apply(long long f, const Journey& j) const override
    {
        if (j.category == Category::Student) return f - pct(f, STUDENT_PERCENT);
        if (j.category == Category::Senior) return f - pct(f, SENIOR_PERCENT);
        return f;
    }
};
class WeekendRule : public Adjustment {
public:
    long long apply(long long f, const Journey& j) const override
    {
        return isWeekend(j.day) ? f - pct(f, WEEKEND_DISCOUNT_PERCENT) : f;
    }
};
class PeakRule : public Adjustment {                                // does not combine with the weekend
    discount
public:
    long long apply(long long f, const Journey& j) const override
    {
        return (!isWeekend(j.day) && isPeak(j.hour, j.minute)) ? f + pct(f, PEAK_SURCHARGE_PERCENT) : f;
    }
};
class FareEngine {
    std::vector<std::unique_ptr<Adjustment>> rules_;
public:
    FareEngine()
    {
        rules_.push_back(std::make_unique<ConcessionRule>());
        rules_.push_back(std::make_unique<WeekendRule>());
        rules_.push_back(std::make_unique<PeakRule>());
    }
    long long fare(const Journey& j) const
    {
        long long f = BASE_PAISE[static_cast<int>(j.zone)];        // table lookup replaces the switch
        for (const auto& r : rules_) f = r->apply(f, j);
        return f;
    }
};
// the same arithmetic without virtual calls, used to measure what the class hierarchy costs
constexpr long long fareDirect(const Journey& j)
{
    long long f = BASE_PAISE[static_cast<int>(j.zone)];
    if (j.category == Category::Student) f -= pct(f, STUDENT_PERCENT);
    else if (j.category == Category::Senior) f -= pct(f, SENIOR_PERCENT);
    if (isWeekend(j.day)) f -= pct(f, WEEKEND_DISCOUNT_PERCENT);
    else if (isPeak(j.hour, j.minute)) f += pct(f, PEAK_SURCHARGE_PERCENT);
    return f;
}
```

The tests are in `t12_revised.cpp`. Expected values were worked out by hand first; for example Airport on Monday 08:00 for a student is 6000 − 1500 = 4500, then 4500 + 900 = 5400.

No.	Case	Expected	Actual	Result
1	Airport Tue 12:00 regular	6000	6000	PASS
2	Airport Tue 09:00 regular (peak +20%)	7200	7200	PASS
3	Inner Sat 12:00 regular (-10%)	900	900	PASS
4	Inner Sat 09:00 regular (peak hour, no surcharge)	900	900	PASS
5	Outer Sun 18:00 regular (no surcharge)	2700	2700	PASS
6	Outer Sat 09:00 student	2025	2025	PASS
7	Airport Sun 12:00 senior	2700	2700	PASS
8	Middle Fri 17:00 senior (peak)	1200	1200	PASS
9	Airport Mon 08:00 student (peak)	5400	5400	PASS
10	pass: Middle journey is free	0	0	PASS
11	pass: balance untouched	10000	10000	PASS
12	pass: Outer not covered, peak fare charged	3600	3600	PASS

No.	Case	Expected	Actual	Result
13	pass: balance after the Outer journey	6400	6400	PASS
14	pass: Inner free even with only Rs 10 on card	0	0	PASS
15	pass: Airport not covered, low balance denied	-1	-1	PASS
16	pass: December journey is not covered	2000	2000	PASS
17	no pass: Airport peak leaves 2800, denied	-1	-1	PASS
18	regression: weekday inputs that differ from v1	0	0	PASS
19	engine vs direct formula: inputs that differ	0	0	PASS

Table 33: Test results for the revised design (paise; −1 means the gate stays closed).

Rows 18 and 19 are not single cases. Row 18 is a **regression check**: for all 12,960 old inputs (3 zones × 24 hours × 60 minutes × 3 categories) on a weekday, the new engine gives the same fare as the original `fare()`, so none of the Task 9 behaviour was lost. Row 19 compares the class hierarchy with a hand-written direct formula for all 120,960 new inputs (4 zones × 7 days × 1,440 minutes × 3 categories); they never differ.

► D. Evidence-based defence

Why modify, not retain or redesign. The table compares the three options on the evidence above:

Option	What it would mean	Verdict
Retain	Add `Airport` and a case; keep `fare(Zone, bool)` and the old `SmartCard`	Rejected: fails Examples 2 and 3, and the missing-case failure is silent
Modify (chosen)	Keep the core (integer paise, constants, rounding, Rs 50 rule); replace the switch by a table; add rule classes, `Journey` fields and `Pass`	Keeps the Task 9 fare behaviour (regression over 12,960 inputs) and passes 19 of 19 new checks
Redesign	Start again, for example with a rule engine read from data	Not justified: three small changes do not need it, and it would invalidate the tested core

Table 34: Retain, modify or redesign.

Trade-offs and cost. The class hierarchy has a price, which I measured:

Fare function	Time per fare (measured, -O2)	Per 3,000,000 journeys
Original design (`fare_core.h`, 3 zones)	1.93 ns	5.8 ms
Revised, direct formula (no virtual calls)	3.03 ns	9.1 ms
Revised, virtual `Adjustment` rules (chosen)	9.61 ns	28.8 ms

Table 35: Time cost of the revised design (all O(1) per journey).

The virtual rules are about three times slower than the direct formula, but 9.6 ns is still tiny: the whole network day costs less than 0.03 s of fare computation. I chose the class version because adding the next rule (a holiday discount, say) is a new class and one line in `FareEngine`, and the engine and the direct formula were proved equal on all 120,960 inputs, so I can switch to the direct version later without changing any fare. Memory per card grows by one pointer (8 to 16 bytes, from `pass_`). All totals stay `long long`, so the Task 10 result is unchanged.

Risks accepted. (i) Assumptions A7 and A8 (weekend rule and pass coverage) must be confirmed; each is one line (`WeekendRule`, `Pass::covers`). (ii) The pass is held by pointer, so it must outlive the card; a real system would store it inside the card record. (iii) The pass has no price or purchase rule; that is outside this assignment. (iv) The day of the week is supplied by the caller and not computed from a date.

Result of Task 12: the original design could not absorb two of the three changes, the modified design passes 19 of 19 checks including a 12,960-input regression, and the price of the extra flexibility is 9.6 ns per fare.

Overall Conclusion

The report followed one fare program from a hurried first version to a tested design that survived a change of rules. The table summarises the evidence-based decision for each part.

Part	Main evidence	Decision
Objects and rules (Tasks 1, 7)	The hurried version reproduced three defects (10 AM / 8 PM, balance −16, three copies of the fares). The procedural program reached −4599; the class refused the same call.	Six objects; `SmartCard` owns the balance; every rule lives in one place.
Constants, types, conversions (Tasks 2–4)	Macro `500 + 500` gave 1500. `double` drifted by Rs 0.019 at 10^7 additions. `-1 < 1u` was false.	`constexpr` constants, `enum class`, `long long` paise, brace initialisation and `static_cast`.
Expressions and control (Tasks 5, 6)	Predictions matched on all nine lines of the two given programs. The 10:00 and 20:00 edges are off-peak.	Half-open minute windows; `switch` or table for the zone, with a `static_assert`.
Input and tests (Tasks 8, 9)	`cin >>` swallowed two good records after a short line; the getline version kept them. 29 of 29 checks pass; a mutation fails exactly one.	Validate every input; table-driven tests including every boundary.
Whole-network behaviour (Tasks 10, 11)	`int` overflows at journey 1,130,039; `float` is wrong by Rs 147,787; `long long` is exact for 5,699,862,900 paise.	Recommend the `long long` / `constexpr` / class design and `-Werror`; a config file only if fares change often.
New rules (Task 12)	The original cannot absorb the weekend rule or the pass; the revised design passes 19 of 19 checks and a 12,960-input regression.	Modify, not redesign; accept 9.6 ns per fare for a class hierarchy.

Table 36: Summary of evidence and decisions.

Three lessons run through the tasks. First, **let the language enforce the rules**: a private member, an `enum class`, a `static_assert` and a brace initialiser each turned a possible run-time mistake into a compile error. Second, **measure before believing**: I expected a one-paisa fare error in `double` and could not reproduce it, and I expected the class version to cost something and found a difference below 1 ns; but the `float` error and the `int` overflow were both larger than I would have guessed. Third, **a test is only worth the bug it can catch**: the mutation test showed that the boundary rows, and nothing else, protect the 10 AM rule.

Final recommendations

1 Store every amount as `long long` paise; use `double` only for display.

2 Keep every fare, percentage and limit in one `constexpr` block; use `enum class` for zones and categories.

3 Keep the balance private; let `deduct()` enforce the Rs 50 rule.

4 Write peak windows as half-open intervals of minutes since midnight.

5 Brace-initialise, cast explicitly, and build with `-Wall -Wextra -Werror`.

6 Read records by line; reject bad input and continue.

7 Keep a table-driven test with every boundary, and re-run it after every fare revision.

8 For the new rules, use a fare table, small rule classes and a `Pass` class, and keep the regression check.

Limitations of this study

- Timings come from one machine and one compiler; the loops are small, and the compiler may vectorise some of them, so I use them for order of magnitude and not for exact ratios.
- All journeys are synthetic. The concession figures (25% and 50%), the weekend rule and the pass coverage are my assumptions A2, A7 and A8.
- I could not reproduce the “off by a paisa” fare of the scenario with the assumed rates; the risk is shown through running totals instead.
- The SIMATS pseudocode standard was not supplied, so the pseudocode style is my own plain form.

- The tests are mine: they show that the program does what I specified, not that my specification matches the metro's.
- Concurrency, time zones and real card hardware are outside the scope.

Mark-by-mark evidence map

Task (marks)	Where it is addressed	Programs in Appendix A
1 (4)	Objects, UML (Figure 1), principles, rules, procedural comparison	legacy_v0, fare_core.h, bench
2 (3)	Tokens, constant kinds, macro and enum errors, identical assembly	t2_*, f_macro, f_constexpr
3 (4)	Type table (Table 11), error growth (Table 12), overflow	t3_types
4 (4)	Predicted and actual output, conversion table, narrowing errors	t4_*, f_implicit, f_explicit
5 (4)	Expressions, precedence table, predicted and actual output	t5_*
6 (4)	Pseudocode, completed code, boundary run, switch against array	t6_*, t12_old
7 (4)	Two versions, private-member error, public-member alternative	t7_*
8 (4)	Safe input, formatted report, getline alternative	t8_*
9 (4)	29-row test table (Table 24), mutation test, hand total	t9_tests
10 (4)	Totals table (Table 26), overflow, warnings	t10_totals
11 (4)	Recommendation table, revision steps, config alternative	t11_config
12 (8)	A impact, B examples, C revised design and tests (Table 33), D defence	t12_old, fare_v2.h, t12_revised

Table 37: Where each task is addressed.

References

- Stroustrup, B., *The C++ Programming Language*, 4th ed., Addison-Wesley (types, operators, classes, enumerations).
- Lippman, S., Lajoie, J., Moo, B., *C++ Primer*, 5th ed. (control structures, I/O streams, conversions).
- ISO/IEC 14882:2017, *Programming languages – C++* (usual arithmetic conversions, evaluation order, narrowing).
- cppreference.com: `enum class`, `constexpr`, `std::stoi`, `<iomanip>`, `<climits>`, `<limits>` (consulted for behaviour, not copied).
- GCC manual, warning options (`-Wall`, `-Wextra`, `-Wsign-compare`, `-Wswitch`, `-Wsequence-point`, `-Wconversion`).
- IEEE 754-2019, *Standard for Floating-Point Arithmetic* (why 0.1 has no exact binary form).
- Course material, DSA01 – Object-Oriented Programming with C++, SIMATS Engineering, CO1 Assignment 1.

Appendix A – Source Code

Each program was compiled with `g++ -std=c++17 -Wall -Wextra` (the timing program also with `-O2`). The same files are in the repository folders listed in Appendix B. Long lines are wrapped for printing.

`fare_core.h` – Shared by Tasks 3, 6, 8, 9, 10: constants, FareTable, SmartCard, Passenger, validators

```
// fare_core.h - the one place where every fare constant and rule of the 3-zone design lives
#pragma once
#include <climits>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>

enum class Zone : unsigned char { Inner, Middle, Outer };
enum class Category : unsigned char { Regular, Student, Senior };

constexpr int ZONE_COUNT = 3;
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_PERCENT = 25;                                // assumption A2
constexpr int SENIOR_PERCENT = 50;                                 // assumption A2
constexpr long long MIN_BALANCE_PAISE = 5000;                      // Rs 50 must remain
constexpr int PEAK_AM_START = 8 * 60;                              // 08:00 included
constexpr int PEAK_AM_END = 10 * 60;                               // 10:00 excluded
constexpr int PEAK_PM_START = 17 * 60;                             // 17:00 included
constexpr int PEAK_PM_END = 20 * 60;                               // 20:00 excluded
static_assert(sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]) == ZONE_COUNT, "one fare per zone");

class FareTable {
public:
    // percentage of an amount in whole paise, rounded half-up, integer arithmetic only
    static constexpr long long pct(long long amount, int percent) { return (amount * percent + 50) / 100; }

    static constexpr bool isPeak(int hour, int minute)
    {
        const int t = hour * 60 + minute;                          // minutes since midnight
#ifdef MUTANT_PEAK_INCLUSIVE                                       // deliberate bug used in the Task 9
    mutation test
        return (t >= PEAK_AM_START && t <= PEAK_AM_END) || (t >= PEAK_PM_START && t < PEAK_PM_END);
#else
        return (t >= PEAK_AM_START && t < PEAK_AM_END) || (t >= PEAK_PM_START && t < PEAK_PM_END);
#endif
    }
    static constexpr int concessionPercent(Category c)
    {
        switch (c) {
            case Category::Student: return STUDENT_PERCENT;
```

```
            case Category::Senior: return SENIOR_PERCENT;
            case Category::Regular: return 0;
        }
        return 0;
    }
    static constexpr long long fare(Zone z, int hour, int minute, Category c = Category::Regular)
    {
        long long f = BASE_PAISE[static_cast<int>(z)];
        f -= pct(f, concessionPercent(c));                          // concession first
        if (isPeak(hour, minute)) f += pct(f, PEAK_SURCHARGE_PERCENT);   // then the surcharge
        return f;
    }
};

class SmartCard {
    long long balance_;                                             // paise, never visible outside
public:
    explicit SmartCard(long long paise = 0) : balance_(paise < 0 ? 0 : paise) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
    bool deduct(long long f)
    {
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
    bool recharge(long long amount)
    {
        if (amount <= 0) return false;
        balance_ += amount;
        return true;
    }
};

class Passenger {
    std::string name_;
    Category category_;
    SmartCard card_;
public:
    Passenger(std::string name, Category c, long long paise) : name_(std::move(name)), category_(c),
        card_(paise) {}
    const std::string& name() const { return name_; }
    Category category() const { return category_; }
    SmartCard& card() { return card_; }
    const SmartCard& card() const { return card_; }
};

struct Journey {
    Zone zone;
    int hour;
    int minute;
};

// ---- validation helpers shared by the gate program and the tests ----
inline bool toZone(int v, Zone& z)
{
    if (v < 0 || v >= ZONE_COUNT) return false;
    z = static_cast<Zone>(v);
    return true;
}
inline bool toCategory(int v, Category& c)
{
    if (v < 0 || v > 2) return false;
    c = static_cast<Category>(v);
    return true;
}
inline bool validTime(int h, int m) { return h >= 0 && h <= 23 && m >= 0 && m <= 59; }

inline bool parseInt(const std::string& s, int& out)               // whole string must be one integer
{
    std::istringstream in(s);
    long long v;
    in >> v;
    if (in.fail()) return false;
    in >> std::ws;
    if (!in.eof()) return false;
    if (v < INT_MIN || v > INT_MAX) return false;
    out = static_cast<int>(v);
    return true;
}

// money is printed from integer paise, so no floating point is ever involved
inline std::string money(long long paise)
{
```

```
    std::ostringstream o;
    if (paise < 0) { o << '-'; paise = -paise; }
    o << paise / 100 << '.' << std::setw(2) << std::setfill('0') << paise % 100;
    return o.str();
}
```

legacy_v0.cpp – Task 1: the hurried first version and its defects

```
// legacy_v0.cpp - the hurried first version of the gate software, kept only to reproduce its defects
#include <iostream>

double balance = 100.0;                      // rupees stored as double

double fareRs(int zone, int hour, bool student)
{
    double f;
    if (zone == 0) f = 10; else if (zone == 1) f = 20; else f = 30;   // fare typed here
    if (student) f = f - f * 0.25;
    if ((hour >= 8 && hour <= 10) || (hour >= 17 && hour < 20)) f = f + f * 0.2;
    return f;
}
long long toPaise(double rs) { return (long long)(rs * 100); }        // truncates

void showTariff()                                                      // fare typed here
{
    std::cout << "Inner Rs 10, Middle Rs 20, Outer Rs 30\n";
}
double revenue(int inner, int middle, int outer)                       // fare typed here
{
    return inner * 10 + middle * 20 + outer * 30;
}
void gate(int zone, int hour, bool student)
{
    double f = fareRs(zone, hour, student);
    if (balance > 0) balance -= f;           // only checks that something is left, not that the fare fits
}

int main()
{
    showTariff();
    std::cout << "revenue(2,1,1) = " << revenue(2, 1, 1) << '\n';
    // defect 2: the two ends of the peak window are treated differently
    std::cout << "10:00 peak? " << (fareRs(0, 10, false) > 10) << "   20:00 peak? "
              << (fareRs(0, 20, false) > 10) << '\n';
    // defect 1: paise lost in double arithmetic
    int bad = 0;
    for (int z = 0; z < 3; ++z)
        for (int h = 0; h < 24; ++h)
            for (int s = 0; s < 2; ++s) {
                double f = fareRs(z, h, s);
                long long exactPaise = (long long)(f * 100 + 0.5);
                if (toPaise(f) != exactPaise) {
                    if (bad < 3)
                        std::cout << "zone " << z << " hour " << h << " student " << s
                                  << ": truncated " << toPaise(f) << " paise, rounded " << exactPaise << '\n';
                    ++bad;
                }
            }
    std::cout << "single fares off by a paisa: " << bad << " of 144\n";
    // defect 3: negative balance
    balance = 20.0;
    gate(2, 18, false);                      // Outer at peak = Rs 36, but only Rs 20 on the card
    std::cout << "balance after an Outer peak journey from Rs 20 = " << balance << '\n';
}
```

t2_constants.cpp – Task 2: macro trap, constexpr, enum and enum class

```
// t2_constants.cpp - macro, constexpr, enum and enum class compared
#include <iostream>

#define FARE_INNER_MACRO 500 + 500           // no brackets: the classic trap
constexpr int FARE_INNER_C = 500 + 500;

enum OldZone { OInner, OMiddle, OOuter };
enum class Zone : unsigned char { Inner, Middle, Outer };

int main()
{
    std::cout << "macro      FARE_INNER_MACRO * 2 = " << FARE_INNER_MACRO * 2 << "  (meant 2000)\n";
    std::cout << "constexpr  FARE_INNER_C * 2     = " << FARE_INNER_C * 2 << '\n';
    int a = OOuter + 5;                      // plain enum silently becomes int
    std::cout << "plain enum: OOuter + 5 = " << a << ", OInner == 0 is " << (OInner == 0) << '\n';
    Zone z = Zone::Outer;
    std::cout << "enum class: Zone::Outer needs a cast to print: " << static_cast<int>(z) << '\n';
    std::cout << "sizeof(OldZone) = " << sizeof(OldZone) << ", sizeof(Zone) = " << sizeof(Zone)
              << ", sizeof(FARE_INNER_C) = " << sizeof(FARE_INNER_C) << '\n';
}
```

t2_err_enum.cpp – Task 2: enum class refuses int

```
enum class Zone { Inner, Middle, Outer };
int x = Zone::Outer;                         // not allowed: no implicit conversion
int main() { return x; }
```

t2_err_macro.cpp – Task 2: macro has no scope

```
#define PEAK 20                              // macro has no scope
struct Rules { int PEAK; };                  // becomes: struct Rules { int 20; };
int main() { Rules r{}; return r.PEAK; }
```

t3_types.cpp – Task 3: sizes, ranges, precision and error growth

```
// t3_types.cpp - sizes, ranges and the precision problem of double money
#include "fare_core.h"
#include <cfloat>
#include <cstdio>
#include <limits>

template <class T> std::string lim(T v)
{
    std::ostringstream o;
    if (std::numeric_limits<T>::is_integer) o << static_cast<long long>(v);
    else o << std::scientific << std::setprecision(3) << v;
    return o.str();
}
template <class T> void row(const char* name, const char* use)
{
    std::cout << std::left << std::setw(16) << name << std::setw(3) << sizeof(T) << std::setw(23)
              << lim(std::numeric_limits<T>::lowest()) << std::setw(23) << lim(std::numeric_limits<T>::max())
              << use << '\n';
}
int main()
{
    std::cout << std::left << std::setw(16) << "type" << std::setw(3) << "B" << std::setw(23) << "min"
              << std::setw(23) << "max" << "used for\n";
    row<bool>("bool", "peak flag");
    row<char>("char", "category letter");
    row<unsigned char>("unsigned char", "Zone, Category (enum base)");
    row<int>("int", "hour, minute, zone number");
    row<unsigned int>("unsigned int", "(not used for money)");
    row<long long>("long long", "fare, balance, totals (paise)");
    row<float>("float", "(only in the experiments)");
    row<double>("double", "(only in the experiments)");
    std::cout << "\nsizeof(SmartCard)=" << sizeof(SmartCard) << " sizeof(Journey)=" << sizeof(Journey)
              << " sizeof(Category)=" << sizeof(Category) << " sizeof(Zone)=" << sizeof(Zone) << '\n';
    std::cout << "INT_MAX=" << INT_MAX << " LLONG_MAX=" << LLONG_MAX << '\n';
    auto f = 7 / 2.0;                                          // auto picks double
    std::cout << "auto f = 7 / 2.0 has sizeof " << sizeof(f) << " and value " << f << '\n';

    std::cout << "\n-- precision of double rupees --\n";
    std::cout << std::setprecision(17);
    std::cout << "0.1 + 0.2        = " << 0.1 + 0.2 << '\n';
    std::cout << "10.8 is stored as " << 10.8 << '\n';
    double s = 0;
```

```
    for (int i = 0; i < 10; ++i) s += 0.10;
    std::cout << "ten times 0.10   = " << s << "  equals 1.00? " << std::boolalpha << (s == 1.0) << '\n';
    std::cout << "3 * 0.1 == 0.3?  " << (3 * 0.1 == 0.3) << '\n';

    std::cout << "\n-- error of a double running total (Rs) against the exact paise total --\n";
    std::cout << std::setprecision(4) << std::scientific;
    const long long marks[] = {10, 1000, 100000, 1000000, 3000000, 10000000};
    for (int which = 0; which < 2; ++which) {
        const double fareRs = which == 0 ? 10.80 : 0.10;
        const long long paise = which == 0 ? 1080 : 10;
        double sum = 0;
        long long n = 0;
        std::cout << "fare Rs " << (which == 0 ? "10.80" : "0.10") << " each:\n";
        for (long long mk : marks) {
            while (n < mk) { sum += fareRs; ++n; }
            const double exact = static_cast<double>(paise * n) / 100.0;
            std::cout << "  n=" << std::setw(9) << std::defaultfloat << n << std::scientific
                      << "  error = " << sum - exact << '\n';
        }
    }
    std::cout << std::defaultfloat << "\nnetwork day: 3000000 journeys x Rs 19.00 average = "
              << 3000000LL * 1900 << " paise; fits in int? " << (3000000LL * 1900 <= INT_MAX) << '\n';
}
```

t4_predict.cpp – Task 4: given program

```
#include <iostream>
int main()
{
    int a = 7, b = 2;
    double c = a / b; double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';
    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';
    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

t4_fix.cpp – Task 4: explicit conversions

```
// t4_fix.cpp - the same four lines written with explicit conversions and brace initialisation
#include <iostream>
int main()
{
    const int a{7}, b{2};
    const double c{static_cast<double>(a) / b};                    // 3.5, not 3
    std::cout << c << '\n';
    const char ch{static_cast<char>('A' + 2)};
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';
    const bool flag{5 != 0};                                       // say what is meant
    std::cout << std::boolalpha << flag << '\n';
    const unsigned int u{1};
    const int s{-1};
    std::cout << (static_cast<long long>(s) < static_cast<long long>(u) ? "s < u" : "s >= u") << '\n';
}
```

t4_narrow.cpp – Task 4: narrowing errors

```
int main()
{
    int x{3.7};                                // narrowing double -> int
    bool f{5};                                 // narrowing int -> bool
    return x + f;
}
```

t4_silent.cpp – Task 4: silent narrowing

```
#include <iostream>
int main()
{
    int x = 3.7;                               // compiles; -Wall -Wextra stay silent
    int fareRs = 30;
    int paise = fareRs * 1.2 * 100;            // double result cut back to int
    std::cout << x << ' ' << paise << '\n';
}
```

t5_predict.cpp – Task 5: given program

```
#include <iostream>
int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

t5_inc.cpp – Task 5: x++, ++x, i = i++

```
#include <iostream>
int main()
{
    int i = 5;
    i = i++;                                // the mistake
    std::cout << "i = i++  gives " << i << '\n';
    int j = 5;
    int k = j++;                            // k gets the old value
    int m = ++j;                            // m gets the new value
    std::cout << "k=" << k << " m=" << m << " j=" << j << '\n';
}
```

t6_complete.cpp – Task 6: completed program (TODO 1 and 2)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {                                                    // TODO 1
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += (base * PEAK_SURCHARGE_PERCENT + 50) / 100;   // integer paise, half-up
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;                        // TODO 2
}

int main()
{
    long long balance = 10000; // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)"; }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

t6_boundary.cpp – Task 6: 10 AM and 8 PM edges

```
// t6_boundary.cpp - the extra journeys asked for in Task 6: exactly 10 AM and exactly 8 PM
#include "fare_core.h"
int main()
{
    struct T { const char* label; int h, m; };
    const T times[] = {{"07:59", 7, 59}, {"08:00", 8, 0},  {"09:59", 9, 59}, {"10:00 (10 AM)", 10, 0},
                       {"16:59", 16, 59}, {"17:00", 17, 0}, {"19:59", 19, 59}, {"20:00 (8 PM)", 20, 0}};
    for (const T& t : times)
        std::cout << std::left << std::setw(15) << t.label << (FareTable::isPeak(t.h, t.m) ? "peak      " :
            "off-peak  ")
                  << "Inner fare Rs " << money(FareTable::fare(Zone::Inner, t.h, t.m)) << '\n';
}
```

t6_alt_array.cpp – Task 6: array lookup alternative

```
// t6_alt_array.cpp - the alternative: base fare from an array indexed by the zone, no switch
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr long long FARE[] = {1000, 2000, 3000};
long long fare(Zone z, bool peak)
{
    long long base = FARE[static_cast<int>(z)];                    // no switch at all
    if (peak) base += (base * 20 + 50) / 100;
    return base;
}
int main()
{
    std::cout << fare(Zone::Inner, false) << ' ' << fare(Zone::Outer, true) << ' ' << fare(Zone::Middle, false)
        << '\n';
    // the danger: a zone value outside 0..2 reads past the array and the compiler says nothing
    std::cout << "index 3 is accepted by the compiler: " << sizeof(FARE) / sizeof(FARE[0]) << " fares exist\n";
}
```

t7_proc.cpp – Task 7: procedural version

```
// t7_proc.cpp - procedural style: global data and free functions
#include <iostream>
long long balance = 10000;                    // global: every function can change it
const long long BASE[] = {1000, 2000, 3000};
const long long MIN_BAL = 5000;

long long fareOf(int zone, bool peak)
{
    long long f = BASE[zone];
    if (peak) f += (f * 20 + 50) / 100;
    return f;
}
bool canTravel(long long f) { return balance - f >= MIN_BAL; }
void deduct(long long f) { balance -= f; }    // nothing forces the caller to ask canTravel() first

int main()
{
    const int zones[] = {0, 2, 1};
    const bool peaks[] = {false, true, false};
    for (int i = 0; i < 3; ++i) {
        long long f = fareOf(zones[i], peaks[i]);
        if (canTravel(f)) { deduct(f); std::cout << "paid " << f << "  balance " << balance << '\n'; }
        else std::cout << "denied   balance " << balance << '\n';
    }
    deduct(9999);                             // a perfectly legal call
    std::cout << "after a careless deduct(9999): " << balance << '\n';
}
```

t7_oo.cpp – Task 7: object-oriented version

```
// t7_oo.cpp - object-oriented style: SmartCard owns its balance
#include <iostream>
class FareTable {
    static constexpr long long BASE[3] = {1000, 2000, 3000};
public:
    static long long fare(int zone, bool peak)
    {
        long long f = BASE[zone];
        if (peak) f += (f * 20 + 50) / 100;
        return f;
    }
};
class SmartCard {
    long long balance_;
    static constexpr long long MIN_BAL = 5000;
public:
    explicit SmartCard(long long b) : balance_(b) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return balance_ - f >= MIN_BAL; }
    bool deduct(long long f)
    {
        if (!canTravel(f)) return false;      // the rule lives here, once
        balance_ -= f;
        return true;
    }
};
int main()
{
    SmartCard card(10000);
    const int zones[] = {0, 2, 1};
    const bool peaks[] = {false, true, false};
    for (int i = 0; i < 3; ++i) {
        long long f = FareTable::fare(zones[i], peaks[i]);
        if (card.deduct(f)) std::cout << "paid " << f << "  balance " << card.balance() << '\n';
        else std::cout << "denied   balance " << card.balance() << '\n';
    }
    bool ok = card.deduct(9999);
    std::cout << std::boolalpha << "careless deduct(9999) accepted? " << ok << ", balance " << card.balance()
        << '\n';
}
```

t7_public.cpp – Task 7: public data member

```
// t7_public.cpp - the alternative: a public data member
#include <iostream>
struct SmartCard { long long balance; };
int main()
{
    SmartCard card{10000};
    card.balance -= 9999;                     // compiles, no Rs 50 check
    std::cout << "balance after careless -=  : " << card.balance << '\n';
    card.balance = -500;                      // compiles, negative balance
    std::cout << "balance after assignment  : " << card.balance << '\n';
}
```

t7_err_private.cpp – Task 7: private member refused

```
#include "fare_core.h"
int main()
{
    SmartCard card(10000);
    card.balance_ = -500;                     // the compiler refuses: balance_ is private
    return 0;
}
```

t8_gate.cpp – Task 8: gate program with cin

```
// t8_gate.cpp - gate program that reads journeys with cin and survives bad input
#include "fare_core.h"
#include <limits>

static void dropLine()
{
    std::cin.clear();                                           // 1. leave the fail state
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');   // 2. throw the bad line away
}

int main()
{
    Passenger p("Saran", Category::Regular, 10000);              // Rs 100.00 on the card
    int n = 0;
    std::cout << "How many journeys? ";
    while (!(std::cin >> n) || n < 0) {
        if (std::cin.eof()) return 1;
        std::cout << "\n  that is not a valid count, try again\nHow many journeys? ";
        dropLine();
    }
    std::cout << "\n\n" << std::left << std::setw(4) << "No" << std::setw(7) << "Zone" << std::setw(8) << "Time"
              << std::right << std::setw(10) << "Fare(Rs)" << std::setw(14) << "Balance(Rs)" << '\n'
              << std::string(43, '-') << '\n';
    for (int i = 1; i <= n; ++i) {
        int zi, h, m, ci;
        if (!(std::cin >> zi >> h >> m >> ci)) {                  // letters instead of numbers, or end of
            input
            if (std::cin.eof()) { std::cout << "input ended early\n"; break; }
            std::cout << std::left << std::setw(4) << i << "rejected: not a number\n";
            dropLine();
            continue;
        }
        Zone z;
        Category c;
        if (!toZone(zi, z) || !validTime(h, m) || !toCategory(ci, c)) {
            std::cout << std::left << std::setw(4) << i << "rejected: value out of range\n";
            continue;
        }
        const long long f = FareTable::fare(z, h, m, c);
        std::cout << std::left << std::setw(4) << i << std::setw(7) << zi << std::setfill('0') << std::right
                  << std::setw(2) << h << ':' << std::setw(2) << m << std::setfill(' ') << "   ";
        if (p.card().deduct(f))
            std::cout << std::setw(10) << money(f) << std::setw(14) << money(p.card().balance()) << '\n';
        else
            std::cout << std::setw(10) << "DENIED" << std::setw(14) << money(p.card().balance()) << '\n';
    }
    // the same value shown with fixed / setprecision, for display only
    std::cout << "\nclosing balance, fixed + setprecision(2): " << std::fixed << std::setprecision(2)
              << p.card().balance() / 100.0 << '\n';
}
```

t8_getline.cpp – Task 8: getline and stoi alternative

```
// t8_getline.cpp - alternative: read a whole line, split it, convert with stoi inside try/catch
#include "fare_core.h"
static bool toInt(const std::string& s, int& out)
{
    try {
        size_t pos = 0;
        const int v = std::stoi(s, &pos);
        if (pos != s.size()) return false;                          // "7x" is not a number
        out = v;
        return true;
    } catch (const std::exception&) {                               // invalid_argument or out_of_range
        return false;
    }
}
int main()
{
    Passenger p("Saran", Category::Regular, 10000);
    std::string line;
    int lineNo = 0, n = -1;
    while (std::getline(std::cin, line)) {
        ++lineNo;
        std::istringstream in(line);
        std::string w[4], extra;
        int got = 0;
        while (got < 4 && in >> w[got]) ++got;
        if (n < 0) {                                                // first line is the count
            if (got == 1 && toInt(w[0], n) && n >= 0) { std::cout << "count " << n << '\n'; }
            else { n = -1; std::cout << "line " << lineNo << ": bad count\n"; }
            continue;
        }
        int v[4];
        if (got != 4 || (in >> extra)) { std::cout << "line " << lineNo << ": need exactly 4 fields\n";
            continue; }
        if (!toInt(w[0], v[0]) || !toInt(w[1], v[1]) || !toInt(w[2], v[2]) || !toInt(w[3], v[3])) {
            std::cout << "line " << lineNo << ": not a number\n";
            continue;
        }
        Zone z; Category c;
        if (!toZone(v[0], z) || !validTime(v[1], v[2]) || !toCategory(v[3], c)) {
            std::cout << "line " << lineNo << ": value out of range\n";
            continue;
        }
        const long long f = FareTable::fare(z, v[1], v[2], c);
        std::cout << "line " << lineNo << ": " << (p.card().deduct(f) ? "paid Rs " + money(f) :
            std::string("DENIED"))
                  << "  balance Rs " << money(p.card().balance()) << '\n';
    }
}
```

t9_tests.cpp – Task 9: table-driven tests

```
// t9_tests.cpp - table-driven tests: every row has an input and a fare worked out by hand first
#include "fare_core.h"

struct Case { const char* name; Zone zone; int h, m; Category cat; long long expected; };

static int passed = 0, total = 0;
static void report(const std::string& name, const std::string& expected, const std::string& actual)
{
    const bool ok = expected == actual;
    ++total;
    passed += ok;
    std::cout << std::right << std::setw(2) << total << "  " << std::left << std::setw(40) << name <<
        std::setw(10)
              << expected << std::setw(10) << actual << (ok ? "PASS" : "FAIL") << '\n';
}

int main()
{
    const Case cases[] = {
        // normal journeys, each zone, off-peak and peak
        {"Inner off-peak 12:00", Zone::Inner, 12, 0, Category::Regular, 1000},
        {"Middle off-peak 12:00", Zone::Middle, 12, 0, Category::Regular, 2000},
        {"Outer off-peak 12:00", Zone::Outer, 12, 0, Category::Regular, 3000},
        {"Inner peak 09:00", Zone::Inner, 9, 0, Category::Regular, 1200},
        {"Middle peak 18:00", Zone::Middle, 18, 0, Category::Regular, 2400},
        {"Outer peak 08:30", Zone::Outer, 8, 30, Category::Regular, 3600},
        // boundaries of the two peak windows
        {"boundary 07:59 (just before)", Zone::Inner, 7, 59, Category::Regular, 1000},
        {"boundary 08:00 (first peak minute)", Zone::Inner, 8, 0, Category::Regular, 1200},
        {"boundary 09:59 (last peak minute)", Zone::Inner, 9, 59, Category::Regular, 1200},
        {"boundary 10:00 (off-peak)", Zone::Inner, 10, 0, Category::Regular, 1000},
        {"boundary 17:00 (first peak minute)", Zone::Inner, 17, 0, Category::Regular, 1200},
        {"boundary 19:59 (last peak minute)", Zone::Inner, 19, 59, Category::Regular, 1200},
        {"boundary 20:00 (off-peak)", Zone::Inner, 20, 0, Category::Regular, 1000},
        // concessions
        {"student Outer off-peak", Zone::Outer, 12, 0, Category::Student, 2250},
        {"student Inner peak", Zone::Inner, 9, 0, Category::Student, 900},
        {"senior Middle off-peak", Zone::Middle, 12, 0, Category::Senior, 1000},
        {"senior Outer peak", Zone::Outer, 18, 0, Category::Senior, 1800},
    };
    std::cout << "No  Case                                     Expected  Actual    Result\n";
    for (const Case& c : cases)
        report(c.name, std::to_string(c.expected), std::to_string(FareTable::fare(c.zone, c.h, c.m, c.cat)));

    // the Rs 50 rule at its edge
    auto yn = [](bool b) { return std::string(b ? "allowed" : "denied"); };
    report("balance 6000, fare 1000 (leaves 5000)", "allowed", yn(SmartCard(6000).canTravel(1000)));
    report("balance 5999, fare 1000 (leaves 4999)", "denied", yn(SmartCard(5999).canTravel(1000)));
    report("balance 6000, fare 1001 (leaves 4999)", "denied", yn(SmartCard(6000).canTravel(1001)));

    // invalid input
    Zone z;
    int h = 0;
    SmartCard card(8000);
    report("unknown zone 7 rejected", "rejected", yn(toZone(7, z)) == "denied" ? "rejected" : "accepted");
    report("zone 2 accepted", "accepted", toZone(2, z) ? "accepted" : "rejected");
    report("negative recharge -200 rejected", "rejected", card.recharge(-200) ? "accepted" : "rejected");
    report("zero recharge rejected", "rejected", card.recharge(0) ? "accepted" : "rejected");
    report("balance unchanged after bad recharges", "8000", std::to_string(card.balance()));
    report("hour typed as 'ab' rejected", "rejected", parseInt("ab", h) ? "accepted" : "rejected");
    report("hour typed as '9x' rejected", "rejected", parseInt("9x", h) ? "accepted" : "rejected");
    report("hour 25 rejected", "rejected", (parseInt("25", h) && validTime(h, 0)) ? "accepted" : "rejected");

    // five journeys, total worked out by hand: 1000 + 2400 + 3600 + 900 + 1000 = 8900
    const Journey js[] = {{Zone::Inner, 7, 59}, {Zone::Middle, 8, 0}, {Zone::Outer, 17, 30}, {Zone::Inner, 9,
        0},
                          {Zone::Middle, 20, 0}};
    const Category cs[] = {Category::Regular, Category::Regular, Category::Regular, Category::Student,
        Category::Senior};
    long long sum = 0;
    for (int i = 0; i < 5; ++i) sum += FareTable::fare(js[i].zone, js[i].hour, js[i].minute, cs[i]);
    report("five journeys, total by hand", "8900", std::to_string(sum));

    std::cout << passed << " of " << total << " checks passed\n";
    return passed == total ? 0 : 1;
}
```

t10_totals.cpp – Task 10: totals in four types

```
// t10_totals.cpp - the same journeys added in long long paise, double rupees, float rupees and int
#include "fare_core.h"
#include <random>

// fare the way the hurried version computed it: rupees in double, concession and surcharge as multipliers
static double legacyFareRs(Zone z, int h, int m, Category c)
{
    double f = 10.0 * (static_cast<int>(z) + 1);
    if (c == Category::Student) f = f - f * 0.25;
    if (c == Category::Senior) f = f - f * 0.50;
    if (FareTable::isPeak(h, m)) f = f + f * 0.2;
    return f;
}
int main()
{
    std::mt19937 rng(2026);                                          // fixed seed: same journeys every run
    const long long checkpoints[] = {20000, 500000, 3000000};
    long long exact = 0, legacyTrunc = 0, mismatch = 0, nPeak = 0;
    double dsum = 0;
    float fsum = 0;
    int isum = 0;
    bool overflowed = false;
    long long firstOverflow = 0;
    unsigned wrapped = 0;
    int next = 0;
    std::cout <<
        "journeys,exact_paise,double_paise,diff,legacy_trunc_paise,legacy_diff,float_rupees,float_err_rupees,int
        _state,int_value\n";
    for (long long i = 1; i <= 3000000; ++i) {
        const Zone z = static_cast<Zone>(rng() % 3);
        const int h = 5 + static_cast<int>(rng() % 18), m = static_cast<int>(rng() % 60);
        const unsigned r = rng() % 10;
        const Category c = r < 7 ? Category::Regular : (r < 9 ? Category::Student : Category::Senior);
        const long long f = FareTable::fare(z, h, m, c);
        const double fd = legacyFareRs(z, h, m, c);
        nPeak += FareTable::isPeak(h, m);
        exact += f;
        dsum += fd;
        fsum += static_cast<float>(fd);
        legacyTrunc += static_cast<long long>(fd * 100);
        if (static_cast<long long>(fd * 100) != f) ++mismatch;
        wrapped += static_cast<unsigned>(f);                         // unsigned arithmetic wraps by definition
        if (!overflowed && __builtin_add_overflow(isum, static_cast<int>(f), &isum)) {
            overflowed = true;
            firstOverflow = i;
        }
        if (i == checkpoints[next]) {
            const long long dpaise = static_cast<long long>(dsum * 100 + 0.5);
            const double ferr = static_cast<double>(fsum) - exact / 100.0;
            std::cout << i << ',' << exact << ',' << dpaise << ',' << dpaise - exact << ',' << legacyTrunc <<
                ','
                      << legacyTrunc - exact << ',' << std::fixed << std::setprecision(0) << fsum << ','
                      << std::setprecision(0) << ferr << std::defaultfloat << ','
                      << (overflowed ? "overflowed" : "fits") << ',' << static_cast<int>(wrapped) << '\n';
            ++next;
        }
    }
    std::cout << "single fares where the hurried formula truncates to a different paisa than fare(): " <<
        mismatch << '\n';
    std::cout << "peak journeys: " << nPeak << " of 3000000\n";
    std::cout << "first journey at which an int total overflows: " << firstOverflow << '\n';
    std::cout << "average fare in paise: " << exact / 3000000 << "." << (exact % 3000000) * 100 / 3000000 <<
        '\n';
}
```

t11_config.cpp – Task 11: fares from a file

```
// t11_config.cpp - alternative: fares read from a file when the gate starts
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <vector>
std::vector<long long> loadFares(const std::string& path)
{
    std::ifstream in(path);
    std::vector<long long> v;
    long long x;
    while (in >> x) {
        if (x <= 0) throw std::runtime_error("fare must be positive");   // a bad file must be caught at run
            time
        v.push_back(x);
    }
    if (v.empty()) throw std::runtime_error("no fares found in " + path);
    return v;
}
int main()
{
    const std::vector<long long> fares = loadFares("fares.cfg");
    for (size_t z = 0; z < fares.size(); ++z) std::cout << "zone " << z << ": " << fares[z] << " paise\n";
    try { loadFares("missing.cfg"); } catch (const std::exception& e) { std::cout << "caught: " << e.what() <<
        '\n'; }
}
```

t12_old.cpp – Task 12: original design with the new requirements

```
// t12_old.cpp - what happens when the ORIGINAL switch design is given the new requirements
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport };                // only the enum was updated

long long fare(Zone z, bool peak)                                 // the Task 6 function, untouched
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = 1000; break;
        case Zone::Middle: base = 2000; break;
        case Zone::Outer:  base = 3000; break;
    }
    if (peak) base += (base * 20 + 50) / 100;
    return base;
}
class SmartCard {                                                 // the Task 7 class, untouched
    long long balance_;
public:
    explicit SmartCard(long long b) : balance_(b) {}
    long long balance() const { return balance_; }
    bool deduct(long long f)
    {
        if (balance_ - f < 5000) return false;
        balance_ -= f;
        return true;
    }
};
int main()
{
    std::cout << "Example 1  Airport off-peak fare: " << fare(Zone::Airport, false) << " paise (should be
        6000)\n";
    std::cout << "Example 2  Outer, Saturday 09:00, student: old code gives " << fare(Zone::Outer, true)
              << " paise (new rules: 2025)\n";
    SmartCard card(1000);                                         // Rs 10.00, holds an Inner+Middle pass in
        the new rules
    std::cout << "Example 3  Middle journey with a pass, balance Rs 10.00: "
              << (card.deduct(0) ? "accepted" : "refused") << " when asked to deduct 0, "
              << (card.deduct(2000) ? "accepted" : "refused") << " when asked to deduct the normal fare\n";
}
```

fare_v2.h – Task 12: revised design

```
// fare_v2.h - revised design: four zones, weekend discount, monthly pass (Task 12)
#pragma once
#include <memory>
#include <vector>

namespace v2 {

enum class Zone : unsigned char { Inner, Middle, Outer, Airport, Count };
enum class Category : unsigned char { Regular, Student, Senior };
enum class Day : unsigned char { Mon, Tue, Wed, Thu, Fri, Sat, Sun };

constexpr int ZONE_COUNT = static_cast<int>(Zone::Count);
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000, 6000};   // Rs 10, 20, 30, 60
static_assert(sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]) == ZONE_COUNT, "one fare per zone");
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;                        // new
constexpr int STUDENT_PERCENT = 25;
constexpr int SENIOR_PERCENT = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;

struct Journey {
    Zone zone;
    int hour, minute;
    Day day;
    Category category;
    int year, month;                                                // needed to test a monthly pass
};

constexpr long long pct(long long a, int p) { return (a * p + 50) / 100; }
constexpr bool isWeekend(Day d) { return d == Day::Sat || d == Day::Sun; }
constexpr bool isPeak(int h, int m)
{
    const int t = h * 60 + m;
    return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
}

// ---- each pricing rule is a small class; the engine just applies them in order ----
class Adjustment {
public:
    virtual ~Adjustment() = default;
    virtual long long apply(long long fare, const Journey& j) const = 0;
};
class ConcessionRule : public Adjustment {
public:
    long long apply(long long f, const Journey& j) const override
    {
        if (j.category == Category::Student) return f - pct(f, STUDENT_PERCENT);
        if (j.category == Category::Senior) return f - pct(f, SENIOR_PERCENT);
        return f;
    }
};
class WeekendRule : public Adjustment {
public:
    long long apply(long long f, const Journey& j) const override
    {
        return isWeekend(j.day) ? f - pct(f, WEEKEND_DISCOUNT_PERCENT) : f;
    }
};
class PeakRule : public Adjustment {                                // does not combine with the weekend discount
public:
    long long apply(long long f, const Journey& j) const override
    {
        return (!isWeekend(j.day) && isPeak(j.hour, j.minute)) ? f + pct(f, PEAK_SURCHARGE_PERCENT) : f;
    }
};
class FareEngine {
    std::vector<std::unique_ptr<Adjustment>> rules_;
public:
    FareEngine()
    {
        rules_.push_back(std::make_unique<ConcessionRule>());
        rules_.push_back(std::make_unique<WeekendRule>());
        rules_.push_back(std::make_unique<PeakRule>());
    }
    long long fare(const Journey& j) const
    {
        long long f = BASE_PAISE[static_cast<int>(j.zone)];         // table lookup replaces the switch
        for (const auto& r : rules_) f = r->apply(f, j);
        return f;
    }
};
// the same arithmetic without virtual calls, used to measure what the class hierarchy costs
constexpr long long fareDirect(const Journey& j)
{
```

```
    long long f = BASE_PAISE[static_cast<int>(j.zone)];
    if (j.category == Category::Student) f -= pct(f, STUDENT_PERCENT);
    else if (j.category == Category::Senior) f -= pct(f, SENIOR_PERCENT);
    if (isWeekend(j.day)) f -= pct(f, WEEKEND_DISCOUNT_PERCENT);
    else if (isPeak(j.hour, j.minute)) f += pct(f, PEAK_SURCHARGE_PERCENT);
    return f;
}

class Pass {                                                    // unlimited journeys, two chosen zones, one
    month
    unsigned mask_;
    int year_, month_;
public:
    Pass(Zone a, Zone b, int year, int month)
        : mask_((1u << static_cast<int>(a)) | (1u << static_cast<int>(b))), year_(year), month_(month) {}
    bool covers(const Journey& j) const
    {
        return j.year == year_ && j.month == month_ && ((mask_ >> static_cast<int>(j.zone)) & 1u) != 0;
    }
};

class SmartCard {
    long long balance_;
    const Pass* pass_ = nullptr;
public:
    explicit SmartCard(long long paise) : balance_(paise) {}
    void attach(const Pass* p) { pass_ = p; }
    long long balance() const { return balance_; }
    // returns the amount charged, or -1 if the gate must stay closed
    long long travel(const Journey& j, const FareEngine& engine)
    {
        if (pass_ != nullptr && pass_->covers(j)) return 0;        // covered: nothing deducted, no Rs 50 test
        const long long f = engine.fare(j);
        if (balance_ - f < MIN_BALANCE_PAISE) return -1;
        balance_ -= f;
        return f;
    }
};

}  // namespace v2
```

t12_revised.cpp – Task 12: tests of the revised design

```
// t12_revised.cpp - tests for the revised design (four zones, weekend discount, monthly pass)
#include "fare_core.h"       // the ORIGINAL design, used only for the regression check
#include "fare_v2.h"

// short names for the revised types; the original types stay in the global namespace for the regression check
using Z = v2::Zone;
using Cat = v2::Category;
using D = v2::Day;
using v2::FareEngine;
using v2::Pass;
using v2::fareDirect;

static int passed = 0, total = 0;
static void check(const char* name, long long expected, long long actual)
{
    const bool ok = expected == actual;
    ++total;
    passed += ok;
    std::cout << std::right << std::setw(2) << total << "  " << std::left << std::setw(54) << name <<
        std::setw(8)
              << expected << std::setw(8) << actual << (ok ? "PASS" : "FAIL") << '\n';
}
static v2::Journey J(Z z, int h, int m, D d, Cat c, int mo = 11) { return {z, h, m, d, c, 2026, mo}; }

int main()
{
    const FareEngine e;
    std::cout << "No  Case                                                  Expected Actual  Result\n";
    check("Airport Tue 12:00 regular", 6000, e.fare(J(Z::Airport, 12, 0, D::Tue, Cat::Regular)));
    check("Airport Tue 09:00 regular (peak +20%)", 7200, e.fare(J(Z::Airport, 9, 0, D::Tue, Cat::Regular)));
    check("Inner Sat 12:00 regular (-10%)", 900, e.fare(J(Z::Inner, 12, 0, D::Sat, Cat::Regular)));
    check("Inner Sat 09:00 regular (peak hour, no surcharge)", 900, e.fare(J(Z::Inner, 9, 0, D::Sat,
        Cat::Regular)));
    check("Outer Sun 18:00 regular (no surcharge)", 2700, e.fare(J(Z::Outer, 18, 0, D::Sun, Cat::Regular)));
    check("Outer Sat 09:00 student", 2025, e.fare(J(Z::Outer, 9, 0, D::Sat, Cat::Student)));
    check("Airport Sun 12:00 senior", 2700, e.fare(J(Z::Airport, 12, 0, D::Sun, Cat::Senior)));
    check("Middle Fri 17:00 senior (peak)", 1200, e.fare(J(Z::Middle, 17, 0, D::Fri, Cat::Senior)));
    check("Airport Mon 08:00 student (peak)", 5400, e.fare(J(Z::Airport, 8, 0, D::Mon, Cat::Student)));

    const Pass pass(Z::Inner, Z::Middle, 2026, 11);
    v2::SmartCard a(10000);
    a.attach(&pass);
```

```
    check("pass: Middle journey is free", 0, a.travel(J(Z::Middle, 9, 0, D::Tue, Cat::Regular), e));
    check("pass: balance untouched", 10000, a.balance());
    check("pass: Outer not covered, peak fare charged", 3600, a.travel(J(Z::Outer, 9, 0, D::Tue, Cat::Regular),
        e));
    check("pass: balance after the Outer journey", 6400, a.balance());
    v2::SmartCard low(1000);
    low.attach(&pass);
    check("pass: Inner free even with only Rs 10 on card", 0, low.travel(J(Z::Inner, 12, 0, D::Tue,
        Cat::Regular), e));
    check("pass: Airport not covered, low balance denied", -1, low.travel(J(Z::Airport, 12, 0, D::Tue,
        Cat::Regular), e));
    v2::SmartCard dec(10000);
    dec.attach(&pass);
    check("pass: December journey is not covered", 2000, dec.travel(J(Z::Middle, 12, 0, D::Tue, Cat::Regular,
        12), e));
    v2::SmartCard nopass(10000);
    check("no pass: Airport peak leaves 2800, denied", -1, nopass.travel(J(Z::Airport, 9, 0, D::Tue,
        Cat::Regular), e));

    // regression: on weekdays the new engine must agree with the original fare() for every old input
    long long checked = 0, different = 0, engineVsDirect = 0;
    for (int z = 0; z < 3; ++z)
        for (int h = 0; h < 24; ++h)
            for (int m = 0; m < 60; ++m)
                for (int c = 0; c < 3; ++c) {
                    const long long oldFare = FareTable::fare(static_cast<::Zone>(z), h, m,
                        static_cast<::Category>(c));
                    const long long newFare = e.fare(J(static_cast<Z>(z), h, m, D::Wed, static_cast<Cat>(c)));
                    ++checked;
                    if (oldFare != newFare) ++different;
                }
    // and the class hierarchy must equal the direct formula for every new input as well
    long long all = 0;
    for (int z = 0; z < v2::ZONE_COUNT; ++z)
        for (int d = 0; d < 7; ++d)
            for (int h = 0; h < 24; ++h)
                for (int m = 0; m < 60; ++m)
                    for (int c = 0; c < 3; ++c) {
                        const v2::Journey j = J(static_cast<Z>(z), h, m, static_cast<D>(d),
                            static_cast<Cat>(c));
                        ++all;
                        if (e.fare(j) != fareDirect(j)) ++engineVsDirect;
                    }
    check("regression: weekday inputs that differ from v1", 0, different);
    std::cout << "    (" << checked << " old inputs compared)\n";
    check("engine vs direct formula: inputs that differ", 0, engineVsDirect);
    std::cout << "    (" << all << " new inputs compared)\n";
    std::cout << passed << " of " << total << " checks passed\n";
    return passed == total ? 0 : 1;
}
```

f_macro.cpp – Task 2: assembly comparison (macro)

```
#define BASE 1000
#define SURCHARGE 20
long long fareM(bool peak) { long long f = BASE; if (peak) f += (f * SURCHARGE + 50) / 100; return f; }
```

f_constexpr.cpp – Task 2: assembly comparison (constexpr)

```
constexpr long long BASE = 1000;
constexpr int SURCHARGE = 20;
long long fareM(bool peak) { long long f = BASE; if (peak) f += (f * SURCHARGE + 50) / 100; return f; }
```

f_implicit.cpp – Task 4: assembly comparison (implicit)

```
double ratio(int a, int b) { double q = a; return q / b; }
```

f_explicit.cpp – Task 4: assembly comparison (explicit)

```
double ratio(int a, int b) { double q{static_cast<double>(a)}; return q / static_cast<double>(b); }
```

bench.cpp – All timings (g++ -O2)

```
// bench.cpp - every timing quoted in the report (best of 5 runs, g++ -O2)
#include "fare_core.h"
#include "fare_v2.h"
#include <chrono>
#include <fstream>
#include <random>
#include <vector>

using Clock = std::chrono::steady_clock;
volatile long long sink;

template <class F> double bestNs(F f, double ops, int reps = 5)
{
    double best = 1e18;
    for (int r = 0; r < reps; ++r) {
        const auto t0 = Clock::now();
        f();
        const auto t1 = Clock::now();
        best = std::min(best, std::chrono::duration<double, std::nano>(t1 - t0).count() / ops);
    }
    return best;
}
static void out(const char* k, double v) { std::cout << k << ' ' << std::fixed << std::setprecision(3) << v <<
    '\n'; }

// ---- procedural version: global balance and free functions ----
static long long g_balance = 10000;
static const long long P_BASE[] = {1000, 2000, 3000};
static long long fareProc(int zone, int h, int m, int cat)
{
    long long f = P_BASE[zone];
    if (cat == 1) f -= (f * 25 + 50) / 100; else if (cat == 2) f -= (f * 50 + 50) / 100;
    const int t = h * 60 + m;
    if ((t >= 480 && t < 600) || (t >= 1020 && t < 1200)) f += (f * 20 + 50) / 100;
    return f;
}

int main()
{
    const int N = 4096;
    std::mt19937 rng(7);
    std::vector<int> zone(N), hour(N), minute(N), cat(N);
    for (int i = 0; i < N; ++i) { zone[i] = rng() % 3; hour[i] = rng() % 24; minute[i] = rng() % 60; cat[i] =
        rng() % 3; }
    const long long REPS = 20000;                                   // 20000 * 4096 = 82 million journeys

    // Task 1: procedural against object-oriented, same work per journey
    out("t1_procedural_ns", bestNs([&] {
        g_balance = 10000;
        for (long long r = 0; r < REPS; ++r)
            for (int i = 0; i < N; ++i) {
                const long long f = fareProc(zone[i], hour[i], minute[i], cat[i]);
                if (g_balance - f >= 5000) g_balance -= f; else g_balance += 100000;
            }
        sink = g_balance;
    }, double(REPS) * N));
    out("t1_oo_ns", bestNs([&] {
        SmartCard card(10000);
        for (long long r = 0; r < REPS; ++r)
            for (int i = 0; i < N; ++i) {
                const long long f = FareTable::fare(static_cast<Zone>(zone[i]), hour[i], minute[i],
                    static_cast<Category>(cat[i]));
                if (!card.deduct(f)) card.recharge(100000);
            }
        sink = card.balance();
    }, double(REPS) * N));

    // Task 3 and 10: cost of one addition for each accumulator type
    std::vector<long long> paise(N);
    std::vector<double> rs(N);
    std::vector<float> fl(N);
    std::vector<unsigned> un(N);
    for (int i = 0; i < N; ++i) { paise[i] = 1000 + 100 * (rng() % 30); rs[i] = paise[i] / 100.0; fl[i] =
        float(rs[i]); un[i] = unsigned(paise[i]); }
    const long long AR = 25000;                                     // 25000 * 4096 = 102 million additions
    out("add_longlong_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i <
        N; ++i) s += paise[i]; sink = s; }, double(AR) * N));
    out("add_double_ns", bestNs([&] { double s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) s += rs[i]; sink = (long long)s; }, double(AR) * N));
    out("add_float_ns", bestNs([&] { float s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) s += fl[i]; sink = (long long)s; }, double(AR) * N));
    out("add_int_ns", bestNs([&] { unsigned s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) s += un[i]; sink = s; }, double(AR) * N));
```

```
    // Task 4: implicit against explicit conversions
    out("conv_implicit_ns", bestNs([&] { double s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i <
        N; ++i) { double q = hour[i]; s += q / (minute[i] + 1); } sink = (long long)s; }, double(AR) * N));
    out("conv_explicit_ns", bestNs([&] { double s{0}; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) { double q{static_cast<double>(hour[i])}; s += q / static_cast<double>(minute[i] + 1); } sink =
        (long long)s; }, double(AR) * N));

    // Task 5: peak by expression against a 24-entry table
    bool table[24];
    for (int h = 0; h < 24; ++h) table[h] = (h >= 8 && h < 10) || (h >= 17 && h < 20);
    out("peak_expr_ns", bestNs([&] { long long c = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) c += (hour[i] >= 8 && hour[i] < 10) || (hour[i] >= 17 && hour[i] < 20); sink = c; }, double(AR) *
        N));
    out("peak_table_ns", bestNs([&] { long long c = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i <
        N; ++i) c += table[hour[i]]; sink = c; }, double(AR) * N));

    // Task 6: switch against array lookup
    static constexpr long long FARE[] = {1000, 2000, 3000};
    out("base_switch_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i <
        N; ++i) { long long b = 0; switch (static_cast<Zone>(zone[i])) { case Zone::Inner: b = 1000; break;
        case Zone::Middle: b = 2000; break; case Zone::Outer: b = 3000; break; } s += b; } sink = s; },
        double(AR) * N));
    out("base_array_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i <
        N; ++i) s += FARE[zone[i]]; sink = s; }, double(AR) * N));

    // Task 8: cin-style extraction against getline + stoi, 1,000,000 lines of "zone hour minute category"
    std::string big;
    for (int i = 0; i < 1000000; ++i) big += std::to_string(zone[i % N]) + ' ' + std::to_string(hour[i % N]) +
        ' ' + std::to_string(minute[i % N]) + ' ' + std::to_string(cat[i % N]) + '\n';
    out("read_extract_ms", bestNs([&] { std::istringstream in(big); long long s = 0; int a, b, c, d; while (in
        >> a >> b >> c >> d) s += a + b + c + d; sink = s; }, 1e6) * 1e6 / 1e6 * 1.0);
    out("read_getline_ms", bestNs([&] {
        std::istringstream in(big);
        std::string line;
        long long s = 0;
        while (std::getline(in, line)) {
            size_t pos = 0;
            for (int k = 0; k < 4; ++k) { s += std::stoi(line.substr(pos), &pos) ; }
        }
        sink = s;
    }, 1e6));
    // Task 9: table of t = 1,000,000 test cases
    struct Case { Zone z; int h, m; Category c; long long expected; };
    std::vector<Case> cases;
    cases.reserve(1000000);
    for (int i = 0; i < 1000000; ++i) {
        const Zone z = static_cast<Zone>(zone[i % N]); const Category c = static_cast<Category>(cat[i % N]);
        cases.push_back({z, hour[i % N], minute[i % N], c, FareTable::fare(z, hour[i % N], minute[i % N], c)});
    }
    out("tests_1M_ms", bestNs([&] { long long bad = 0; for (const Case& k : cases) bad += FareTable::fare(k.z,
        k.h, k.m, k.c) != k.expected; sink = bad; }, 1e6) );
    out("test_row_bytes", double(sizeof(Case)));

    // Task 12: original switch design against table + virtual rules against table + direct formula
    std::vector<v2::Journey> js(N);
    for (int i = 0; i < N; ++i) js[i] = {static_cast<v2::Zone>(zone[i] % 4), hour[i], minute[i],
        static_cast<v2::Day>(rng() % 7), static_cast<v2::Category>(cat[i]), 2026, 11};
    const v2::FareEngine engine;
    out("fare_v1_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i < N;
        ++i) s += FareTable::fare(static_cast<Zone>(zone[i]), hour[i], minute[i],
        static_cast<Category>(cat[i])); sink = s; }, double(AR) * N));
    out("fare_v2_virtual_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0;
        i < N; ++i) s += engine.fare(js[i]); sink = s; }, double(AR) * N));
    out("fare_v2_direct_ns", bestNs([&] { long long s = 0; for (long long r = 0; r < AR; ++r) for (int i = 0; i
        < N; ++i) s += v2::fareDirect(js[i]); sink = s; }, double(AR) * N));

    // Task 11: start-up cost of reading the fare table from a file
    { std::ofstream f("/tmp/fares_bench.cfg"); f << "1000\n2000\n3000\n6000\n"; }
    out("config_read_us", bestNs([&] {
        long long s = 0;
        for (int r = 0; r < 20000; ++r) { std::ifstream in("/tmp/fares_bench.cfg"); std::vector<long long> v;
            long long x; while (in >> x) v.push_back(x); s += v.size(); }
        sink = s;
    }, 20000) / 1000.0);
}
```

Appendix B – Repository, Commands and Raw Data

Repository layout. The repository is named `DSA01-192473003-Assignment1` and has one folder per task. Each folder holds the C++ source, any input files, the captured output (`.txt`) and a screenshot of the run.

Folder	Contents
Task01	legacy_v0.cpp, fare_core.h, output of legacy_v0
Task02	t2_constants.cpp, t2_err_enum.cpp, t2_err_macro.cpp, f_macro.cpp, f_constexpr.cpp
Task03	t3_types.cpp, fare_core.h
Task04	t4_predict.cpp, t4_fix.cpp, t4_narrow.cpp, t4_silent.cpp, f_implicit.cpp, f_explicit.cpp
Task05	t5_predict.cpp, t5_inc.cpp
Task06	t6_complete.cpp, t6_boundary.cpp, t6_alt_array.cpp, fare_core.h
Task07	t7_proc.cpp, t7_oo.cpp, t7_public.cpp, t7_err_private.cpp, fare_core.h
Task08	t8_gate.cpp, t8_getline.cpp, t8_a.in, t8_b.in, t8_c.in, fare_core.h
Task09	t9_tests.cpp, fare_core.h
Task10	t10_totals.cpp, fare_core.h
Task11	t11_config.cpp, fares.cfg
Task12	t12_old.cpp, t12_revised.cpp, fare_v2.h, fare_core.h
bench	bench.cpp, fare_core.h, fare_v2.h (all timings)

Table 38: Repository layout (one commit per task).

Commands.

```
g++ -std=c++17 -Wall -Wextra t9_tests.cpp -o t9 && ./t9
g++ -std=c++17 -Wall -Wextra -DMUTANT_PEAK_INCLUSIVE t9_tests.cpp -o t9m && ./t9m    # mutation test
g++ -std=c++17 -Wall -Wextra t8_gate.cpp -o t8 && ./t8 < t8_a.in
g++ -std=c++17 -O2 -Wall -Wextra bench.cpp -o bench && ./bench                       # timings
g++ -std=c++17 -S -O2 f_macro.cpp -o f_macro.s                                       # assembly
    comparison
```

README.md contents. Name: Saran. S; register number 192473003; compiler g++ 13.3.0 (Ubuntu 24.04, x86-64); compile command `g++ -std=c++17 -Wall -Wextra <file>.cpp -o <name>`; run with `./<name>` (Task 8 programs read standard input: `./t8 < t8_a.in`); one line per task with its result, for example “Task 9: 29 of 29 checks pass, mutation test fails 1”.

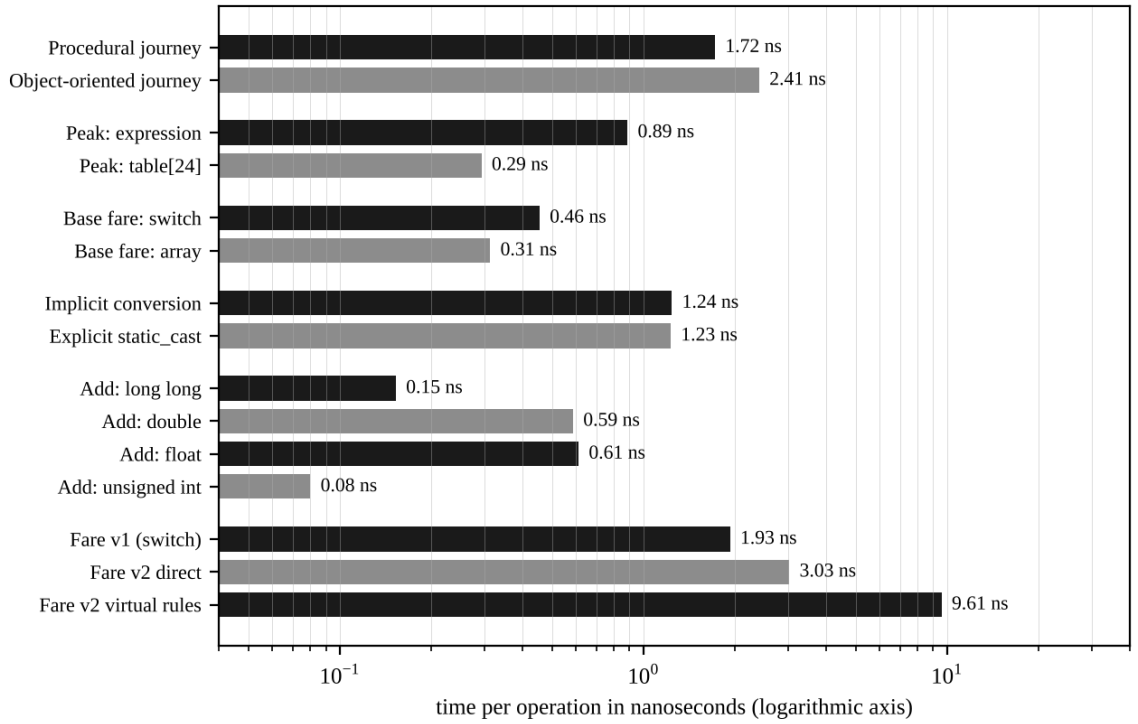


Figure 7: All timing comparisons of the report on one logarithmic axis (g++ -O2, best of 5).

All timings (best of 5, `-O2`, nanoseconds per operation unless the name says otherwise):

Measurement	Value
Procedural journey	1.723
Object-oriented journey	2.409
Add long long	0.153
Add double	0.587
Add float	0.612
Add unsigned int	0.080
Implicit conversion	1.242
Explicit static_cast	1.231
Peak by expression	0.887
Peak by table	0.293
Base fare by switch	0.455
Base fare by array	0.313
cin-style extraction (ms per 1,000,000 lines)	118.988
getline + stoi (ms per 1,000,000 lines)	82.405
1,000,000 table tests (ms)	5.528
Fare v1	1.931
Fare v2 direct	3.027
Fare v2 virtual rules	9.607
Read fares.cfg (µs)	1.288

Table 39: Raw timing results.

Appendix C - Complete C++17 Fare Program

Core three-zone task: paise arithmetic, peak fares, concessions and SmartCard balance rule

```
#include <array>
#include <iomanip>
#include <iostream>
#include <string>
#include <sstream>

enum class Zone { Inner, Middle, Outer };
enum class Category { Regular, Student, Senior };

constexpr std::array<long long, 3> BASE_PAISE{1000, 2000, 3000};
constexpr long long MINIMUM_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_DISCOUNT_PERCENT = 25;
constexpr int SENIOR_DISCOUNT_PERCENT = 50;

long long percentage(long long amount, int rate) {
    return (amount * rate + 50) / 100; // nearest paisa, half up
}

bool validTime(int hour, int minute) {
    return hour >= 0 && hour <= 23 && minute >= 0 && minute <= 59;
}

bool isPeak(int hour, int minute) {
    const int time = hour * 60 + minute;
    return (time >= 8 * 60 && time < 10 * 60) ||
           (time >= 17 * 60 && time < 20 * 60);
}

long long fare(Zone zone, int hour, int minute, Category category) {
    long long amount = BASE_PAISE.at(static_cast<std::size_t>(zone));
    if (category == Category::Student)
        amount -= percentage(amount, STUDENT_DISCOUNT_PERCENT);
    else if (category == Category::Senior)
        amount -= percentage(amount, SENIOR_DISCOUNT_PERCENT);
    if (isPeak(hour, minute))
        amount += percentage(amount, PEAK_SURCHARGE_PERCENT);
    return amount;
}

class SmartCard {
public:
    explicit SmartCard(long long balancePaise) : balance_(balancePaise) {}
    long long balance() const { return balance_; }
    bool deduct(long long amount) {
        if (amount <= 0 || balance_ - amount < MINIMUM_BALANCE_PAISE)
            return false;
        balance_ -= amount;
```

Appendix C - Complete C++17 Fare Program (continued)

Core three-zone task: paise arithmetic, peak fares, concessions and SmartCard balance rule

```
        return true;
    }
private:
    long long balance_; // all money is stored as whole paise
};

std::string money(long long paise) {
    std::ostringstream out;
    out << paise / 100 << '.' << std::setw(2) << std::setfill('0') << paise % 100;
    return out.str();
}

int main() {
    long long startingRupees;
    int journeys;
    std::cout << "Starting balance (rupees) and number of journeys: ";
    if (!(std::cin >> startingRupees >> journeys) || startingRupees < 0 || journeys < 0) {
        std::cerr << "Invalid balance or journey count.\n";
        return 1;
    }
    SmartCard card(startingRupees * 100);

    std::cout << "Enter each journey as: zone(1-3) hour minute category(0-2)\n"
              << "Category: 0 regular, 1 student, 2 senior\n";
    for (int i = 1; i <= journeys; ++i) {
        int zoneInput, hour, minute, categoryInput;
        std::cout << "Journey " << i << ": ";
        if (!(std::cin >> zoneInput >> hour >> minute >> categoryInput)) {
            std::cerr << "Invalid journey input.\n";
            return 1;
        }
        if (zoneInput < 1 || zoneInput > 3 || !validTime(hour, minute) ||
            categoryInput < 0 || categoryInput > 2) {
            std::cout << "Rejected: value out of range.\n";
            continue;
        }

        const auto zone = static_cast<Zone>(zoneInput - 1);
        const auto category = static_cast<Category>(categoryInput);
        const long long amount = fare(zone, hour, minute, category);
        if (card.deduct(amount))
            std::cout << "Fare Rs " << money(amount) << " accepted; balance Rs "
                      << money(card.balance()) << "\n";
        else
            std::cout << "Travel denied; balance Rs " << money(card.balance()) << "\n";
    }
}
```